

PAUTI PUSTAK

Backend System Architecture & Module Requirements

Enterprise-Grade Microservices ProDoc Reference Manual

REVISED EDITION - VOLUNTEERS, PERSONALIZED BILLS, COLLECTIONS & RECEIPTS

Document Type	System Architecture & Module Requirements
Version	2.1 - Enterprise Microservices Revised Edition
Date	23 July 2026
Product	Pauti Pustak - Multi-Tenant Event Financial Management Platform
Primary Focus	Volunteers, personalized contribution bills, payment collections, contribution records, receipts, expenses, balances, public transparency, and reports
Backend Framework	NestJS 11 / TypeScript / Node.js LTS / Nx Monorepo
Database / ORM	PostgreSQL database-per-service / Prisma ORM
Supporting Services	NATS JetStream, gRPC, Redis, BullMQ, S3-compatible storage, Razorpay, Puppeteer, OpenTelemetry, WhatsApp, Email, SMS OTP
Architecture Style	Domain-aligned microservices; independently deployable services and workers in an Nx monorepo

Prepared for MNC-grade implementation, independent service ownership, automated testing, secure deployment, review, financial auditability, and acceptance.

Document Header Block

Document Title	Pauti Pustak Backend System Architecture & Module Requirements
Document Type	Backend System Architecture & Module Requirements ProDoc
Version	2.1 - Enterprise Microservices Revised Edition
Date	23 July 2026
Target System Stack	NestJS 11, TypeScript, Nx/pnpm monorepo, PostgreSQL database-per-service, Prisma ORM, NATS JetStream, gRPC, Redis, BullMQ, private S3-compatible storage, Razorpay, REST/OpenAPI, AsyncAPI, Protobuf, JWT, Argon2id, Handlebars/Puppeteer, Docker, Kubernetes, Helm, OpenTelemetry, Playwright, Pact, Testcontainers, k6
Currency & Precision	INR; monetary values stored as integer paise; no floating-point arithmetic for finance
Languages	English, Marathi, Hindi
Tenancy	Organization-scoped shared SaaS tenancy with service-level tenant enforcement, database isolation boundaries, and PostgreSQL row-level-security defense in depth

Version 2.1 Revision Scope

Supersession Rule

Version 2.1 preserves all non-conflicting Version 2.0 requirements. The updated directory matrix on pages 3-4 and the Version 2.1 domain specifications beginning on page 65 supersede conflicting module names, service ownership, workflows, schemas, APIs, and acceptance criteria.

- Adds a dedicated Volunteer Management bounded context for non-user volunteers, collectors, event assignments, area assignments, and activity history.
- Adds event-specific Contributor Accounts, personalized Contribution Bills, batch bill generation, payment links, and immutable bill snapshots.
- Adds Collection Records with manual verification, cheque clearance, collector settlement, reconciliation, and exact bill allocation.
- Generalizes the existing Donations aggregate into Contribution Records while retaining compatibility for existing donation APIs during migration.
- Extends receipts to include bill number, collector, payment allocation, remaining balance, multilingual rendering, QR verification, cancellation, and replacement.

Financial Formulas

Bill Payable Amount = requestedAmountPaise - waiverAmountPaise
Bill Outstanding Amount = max(payableAmountPaise - confirmedCollectionAmountPaise, 0)
Collector Cash Variance = handedOverCashPaise - confirmedCashCollectionPaise
Confirmed Contribution Total = sum(confirmed collection allocations) + confirmed direct installments

Directory & Task Assignment Matrix

Module File	System/Directory Path	Core Responsibility	Assigned To	Status
auth.md	apps/identity-service/src/modules/auth/	Authentication, JWT sessions, OTP, logout, and global guards.	Unassigned - Security Engineering	Pending
users.md	apps/identity-service/src/modules/users/	Canonical user identity, verified contacts, language, status, and profile administration.	Unassigned - Core Backend	Pending
donors.md	apps/donor-service/src/modules/donors/	Canonical donor/person/business identity, duplicate detection, organization-assisted creation, and merge history.	Unassigned - Core Backend	Pending
contributor-accounts.md	apps/donor-service/src/modules/contributor-accounts/	Event-specific household, individual, shop, society, sponsor, and institution contribution accounts with billing preferences.	Unassigned - Business Domain	Pending
platform-admin.md	apps/platform-admin-service/src/modules/platform-admin/	Platform organization and donor administration without tenant-private finance access.	Unassigned - Platform Operations	Pending
organizations.md	apps/tenant-service/src/modules/organizations/	Organization registration, legal profile, lifecycle, ownership, and immutable organization code.	Unassigned - Core Backend	Pending
memberships.md	apps/tenant-service/src/modules/memberships/	Invitations, accounts, active-organization restriction, status, and ownership transfer.	Unassigned - Access Control	Pending
roles.md	apps/tenant-service/src/modules/roles/	Protected roles, custom roles, permission catalog, and server-side authorization.	Unassigned - Access Control	Pending
events.md	apps/event-service/src/modules/events/	Event creation, lifecycle, closure, reopening, financial status, and immutable event code.	Unassigned - Business Domain	Pending
volunteers.md	apps/volunteer-service/src/modules/volunteers/	Volunteer master records, optional user linkage, privacy controls, status, documents, and history.	Unassigned - Volunteer Operations	Pending
volunteer-assignments.md	apps/volunteer-service/src/modules/assignments/	Event role, collector, area, route, contributor portfolio, availability, and assignment history.	Unassigned - Volunteer Operations	Pending
contributions.md	apps/contribution-service/src/modules/contributions/	Unified monetary contribution pledges, direct collections, bill-linked collections, grants, sponsorships, installments, verification, and confirmation.	Unassigned - Finance Domain	Pending
contribution-bills.md	apps/contribution-service/src/modules/contribution-bills/	Personalized bill creation, batch generation, atomic numbering, immutable snapshots, PDF/QR/payment links, and lifecycle.	Unassigned - Finance Domain	Pending
collection-records.md	apps/contribution-service/src/modules/collection-records/	Cash/UPI/bank/cheque/card/Razorpay collection records, verification, allocations, collector settlement, and reconciliation.	Unassigned - Finance Domain	Pending
receipt-books.md	apps/contribution-service/src/modules/receipt-books/	Multiple event receipt books, optional collector assignment, atomic event-year numbering, and closure.	Unassigned - Finance Domain	Pending

Directory & Task Assignment Matrix - Continued

Module File	System/Directory Path	Core Responsibility	Assigned To	Status
receipts.md	apps/contribution-service/src/modules/receipts/	Immutable multilingual contribution receipts, bill/collection snapshots, PDF, QR, cancellation, replacement, and sharing.	Unassigned - Documents & Finance	Pending
in-kind-contributions.md	apps/contribution-service/src/modules/in-kind-contributions/	Non-cash contribution intake, valuation, receipts, storage/use notes, and separate reporting.	Unassigned - Business Domain	Pending
vendors.md	apps/finance-service/src/modules/vendors/	Organization-scoped vendor records with sensitive data protection and frozen snapshots.	Unassigned - Finance Domain	Pending
approval-policies.md	apps/finance-service/src/modules/approval-policies/	Expense thresholds, sequential approvals, immutable versions, and submission-time snapshots.	Unassigned - Finance Architecture	Pending
expenses.md	apps/finance-service/src/modules/expenses/	Vendor expenses, vouchers, approvals, partial payments, cancellation, and immutable approved records.	Unassigned - Finance Domain	Pending
accounts.md	apps/finance-service/src/modules/accounts/	Cash, bank, UPI, cheque, Razorpay, and card accounts with ledger-backed balances and transfers.	Unassigned - Finance Architecture	Pending
ledger.md	apps/finance-service/src/modules/ledger/	Append-only double-entry source of truth for contributions, expenses, fees, refunds, transfers, and reconciliation.	Unassigned - Finance Architecture	Pending
payments.md	apps/payment-service/src/modules/payments/	Organization-connected Razorpay checkout, webhooks, settlement, fees, reconciliation, and limited refunds.	Unassigned - Payment Integrations	Pending
public-events.md	apps/public-query-service/src/modules/public-events/	Public event pages, contribution visibility, live totals, redacted bills/vouchers, and immutable snapshots.	Unassigned - Public Platform	Pending
reports.md	apps/reporting-service/src/modules/reports/	Event, volunteer, bill, collection, contribution, receipt, expense, account, and audit reports with exports.	Unassigned - Business Intelligence	Pending
notifications.md	apps/notification-service/src/modules/notifications/	In-app, email, WhatsApp, and OTP SMS delivery in English, Marathi, and Hindi.	Unassigned - Integrations	Pending
documents.md	apps/document-service/src/modules/documents/	Private storage, signed access, file validation, malware scanning, retention, and generated documents.	Unassigned - Infrastructure	Pending
audit.md	apps/audit-service/src/modules/audit/	Immutable evidence for volunteer, billing, collection, contribution, receipt, security, and finance actions.	Unassigned - Security & Audit	Pending
settings.md	apps/tenant-service/src/modules/settings/	Versioned language, financial year, receipt/bill templates, collector rules, visibility, and platform limits.	Unassigned - Core Configuration	Pending

Global Non-Functional Requirements

Capacity Targets

- Design target: 10,000 organizations, 100,000 users, 100,000 donations for a large event, and at least 1 million annual financial transactions.
- All list endpoints use indexed pagination; default 25 and maximum 100 records unless an export job is used.

Performance Targets

- Standard API p95 under 500 ms, filtered list p95 under 800 ms, and payment webhook acknowledgment under 2 seconds.
- Receipt PDF under 5 seconds under normal load; standard reports under 3 seconds; large exports run asynchronously.

Reliability & Recovery

- Daily encrypted snapshots plus point-in-time recovery; target RPO 15 minutes and RTO 4 hours.
- Quarterly restore tests, health checks, dead-letter monitoring, reconciliation jobs, and alerting are mandatory.

Deployment & Quality Gates

- Dockerized, non-root, independently deployable microservices and workers run on Kubernetes using Helm; managed PostgreSQL, NATS JetStream, Redis, and private object storage are provisioned per environment.
- CI runs Nx affected lint/type-check/build, Jest unit tests, Testcontainers integration tests, contract compatibility, Playwright API/browser E2E, migration validation, SAST/DAST, dependency/container scans, SBOM generation, and production builds.
- A module/service is not complete without owned migrations, OpenAPI/AsyncAPI or Protobuf contracts, authorization, audit hooks, observability, automated tests, runbooks, operational error codes, dashboards, alerts, and rollback evidence.

Enterprise Architecture Gap Closure Summary

Version 2.0 replaces the earlier modular-monolith deployment assumption with an enterprise microservices operating model while retaining the validated domain requirements and financial controls. The following decisions are mandatory and supersede conflicting Version 1.0 implementation notes.

Gap / Concern	Version 2.0 Enterprise Decision	Acceptance Evidence
Multi-tenant isolation	Retained and strengthened at Gateway, service, repository, message, cache-key, file-path, and database-policy layers.	Automated cross-tenant negative tests, RLS tests, scoped exports/files, and trace evidence.
Microservices architecture	Domain-aligned, independently deployable services with service-owned data; no shared ORM or cross-service SQL.	Separate build/deploy artifacts, databases/Prisma clients, health endpoints, dashboards, and ownership.
Playwright and integration testing	Playwright is mandatory for API E2E and critical browser journeys; Testcontainers and contract tests cover service integration.	CI reports, traces/screenshots, test artifacts, contract verification, and environment teardown.
Docker and production orchestration	Multi-stage non-root images deployed by Helm to managed Kubernetes with probes, autoscaling, disruption budgets, and rollback.	Signed image/SBOM, deployment manifest, readiness/liveness checks, canary evidence, and rollback drill.
Reusable packages and monorepo	Nx + pnpm monorepo with enforceable module boundaries, shared contracts/platform libraries, CODEOWNERS, and affected CI.	Nx dependency graph, boundary linting, package ownership, semantic version policy, and affected-only pipeline.
No hardcoding / configurable behavior	Typed configuration, versioned tenant settings, reference-data catalogs, feature flags, localized templates, and secret management.	Configuration schema tests, audit history, cache invalidation, no magic constants/secrets in code or images.
Enterprise operations	OpenTelemetry observability, SLOs, incident runbooks, security gates, data governance, and architecture decision records.	Dashboards, alerts, runbooks, ADRs, threat models, release evidence, and quarterly recovery tests.

Enterprise Microservices Reference Architecture

Architecture Principles

- Each bounded context is a separate NestJS application and independently deployable container. Repository colocation in Nx does not imply runtime coupling.
- External clients access only the API Gateway. Internal services are private and authenticate through workload identity and mutual TLS.
- Synchronous calls are minimized and use gRPC with deadlines, cancellation, authorization context, and bounded retries. Business notifications and state propagation use durable NATS JetStream events.
- Every service owns its database and migration lifecycle. Cross-service references are opaque UUIDs; joins across databases are replaced by APIs, events, or owned read models.
- Financial workflows use local ACID transactions plus transactional outbox/inbox and saga compensation. Distributed two-phase commit is prohibited.
- Services must be independently scalable, observable, deployable, and reversible without requiring a full-platform release.

Deployable Service Topology

Deployable Service	Owned Modules	Owned Persistence	Primary Interfaces	Scaling Profile
api-gateway	Edge routing, authentication context, quotas, request validation	No business DB; distributed rate-limit state in Redis	Public REST/OpenAPI; internal gRPC	Horizontal by request rate
identity-service	auth, users	identity_db	gRPC; identity/session events	CPU/network; low-latency
tenant-service	organizations, memberships, roles, settings	tenant_db	gRPC; tenant/role/config events	Read-heavy with transactional admin writes
donor-service	donors and duplicate merge	donor_db	gRPC; donor lifecycle events	Search/read-heavy
event-service	events	event_db	gRPC; event lifecycle events	Moderate, tenant-partitioned

Deployable Service	Owned Modules	Owned Persistence	Primary Interfaces	Scaling Profile
contribution-service	donations, receipt-books, receipts, in-kind contributions	contribution_db	REST via gateway, gRPC, contribution/receipt events	Peak festival write load; PDF worker scaling
finance-service	vendors, approval-policies, expenses, accounts, ledger	finance_db	gRPC; ledger/expense/account events	Strict consistency; independently scaled workers
payment-service	Razorpay connections, payments, webhooks, refunds	payment_db	Public webhook ingress, gRPC, payment events	Burst webhook load; idempotent consumers
public-query-service	public event pages and approved snapshots	public_query_db/read model	Public REST; event consumers	High-read/cacheable
reporting-service	reports and exports	reporting_db/read models	REST; NATS consumers; export workers	CPU/memory worker scaling
notification-service	in-app, email, WhatsApp, OTP SMS orchestration	notification_db	NATS consumers; provider adapters	Queue-depth based scaling
document-service	private assets, malware scan, redaction, signed access	document_db + object storage	gRPC; document events; worker queues	I/O and worker scaling
audit-service	append-only security and business evidence	audit_db/WORM archive	NATS audit subjects; restricted query API	Write-heavy append-only
platform-admin-service	platform cases and non-financial operations	platform_admin_db	REST via gateway; gRPC to administrative APIs	Low volume, high-control

Communication and Contract Standards

Interaction	Mandatory Technology	Standard	Prohibited Pattern
Client to platform	REST/JSON through API Gateway	OpenAPI 3.x, /api/v1, idempotency key, correlation ID, OAuth/JWT context	Direct public access to internal services
Synchronous service call	gRPC/Protobuf	Explicit deadlines, mTLS, propagated tenant/user context, backward-compatible fields	Unbounded retries, chatty call chains, shared database reads
Domain event	NATS JetStream	AsyncAPI schema, subject pauti.<domain>.<aggregate>.<event>.>v1, messageId, occurredAt, traceparent	Unversioned JSON events or fire-and-forget finance events
Service-local background job	BullMQ/Redis	Idempotent job key, retry policy, dead-letter queue, bounded concurrency	Using a local queue as a cross-service source of truth
Files and generated documents	Private S3-compatible storage	Checksum, malware state, tenant prefix, short-lived signed URL, immutable finalized object	Public buckets or permanent sensitive URLs
Contract distribution	libs/contracts + generated clients	Semantic versioning, compatibility checks, deprecation window	Copy-pasted DTOs or importing another service internals

Data Ownership, Consistency, and Event Reliability

- Database-per-service is the production rule. A managed PostgreSQL cluster may host multiple service databases, but credentials, schemas, migrations, backups, and access policies remain isolated.
- Cross-service IDs are scalar UUIDs without ORM relations or foreign keys. The owning service validates existence through a contract call or consumes an authoritative lifecycle event.
- Each state-changing service transaction writes the aggregate change and an OutboxMessage in the same local database transaction. A relay publishes the event to JetStream and marks it delivered.
- Every consumer maintains an InboxMessage record keyed by messageId to guarantee idempotent processing and safe redelivery.
- Long-running workflows use explicit sagas. Example: payment verified -> donation confirmed -> ledger posted -> receipt allocated -> document rendered -> notification sent. Each step has retry, timeout, and compensation rules.
- Reporting and public services maintain eventually consistent read models; they do not query operational service databases.
- Distributed locks are avoided for financial correctness. Atomic counters and unique constraints remain inside the owning service database.

Nx Monorepo and Reusable Package Strategy

The source repository uses Nx with pnpm workspaces. A monorepo is a repository and delivery strategy, not a monolithic runtime: every application below has its own build target, image, deployment, ownership, and release cadence.

apps/	
api-gateway/	
identity-service/	
tenant-service/	
donor-service/	
event-service/	
contribution-service/	
finance-service/	
payment-service/	
public-query-service/	
reporting-service/	
notification-service/	
document-service/	
audit-service/	
platform-admin-service/	
e2e/	# Playwright API and browser journeys
libs/	
contracts/	# OpenAPI DTOs, Protobuf, AsyncAPI event schemas
platform/	# error model, pagination, idempotency, tracing, health
security/	# guards, policy interfaces, crypto abstractions
config/	# typed configuration schemas and loaders
observability/	# OpenTelemetry/Pino instrumentation
testing/	# factories, fixtures, Testcontainers harnesses
domain/<service>/	# private libraries usable only by owning service
tools/	# Nx generators, migration checks, release automation
deploy/	
docker/	
helm/	
environments/	
policies/	

- Nx project tags enforce boundaries such as type:app, type:contract, type:platform, scope:finance, and scope:contribution. A service may depend on contracts/platform libraries but never another service implementation.
- Use a single-version dependency policy by default. Exceptions require an ADR, owner, vulnerability plan, and expiry date.
- Every public library exports through one index entry point; deep imports and circular dependencies fail CI.
- Contracts are generated and versioned. Breaking changes require a new API/event version and a documented migration/deprecation period.
- CODEOWNERS protects each service and shared library. Finance, security, identity, and contracts require at least two reviewers, including the owning team.
- Libraries are published to an internal registry only when consumed outside this repository. Internal-only libraries remain workspace packages to avoid unnecessary release overhead.
- Custom Nx generators create services, modules, migrations, event consumers, OpenAPI definitions, tests, Dockerfiles, Helm values, dashboards, and runbook skeletons consistently.

Enterprise Platform Workstream Assignment Matrix

Workstream	Path	Responsibility	Default Owner	Status
API Gateway & Edge Security	apps/api-gateway/	Routing, JWT validation, quotas, WAF integration, request IDs, tenant context	Platform Engineering	Pending
Service Contracts	libs/contracts/	OpenAPI, Protobuf, AsyncAPI, generated clients, compatibility policy	Architecture / API Governance	Pending
Reusable Platform Libraries	libs/platform/, libs/security/, libs/config/, libs/observability/	Cross-cutting primitives without domain orchestration	Platform Engineering	Pending

Workstream	Path	Responsibility	Default Owner	Status
Playwright E2E	apps/e2e/	API and browser journeys, multi-tenant isolation, trace/video evidence	Quality Engineering	Pending
Container & Kubernetes Platform	deploy/docker/, deploy/helm/	Images, Helm, environments, policies, rollout/rollback	DevSecOps / SRE	Pending
Observability & SLOs	libs/observability/, deploy/monitoring/	Tracing, logs, metrics, dashboards, alerts, runbooks	SRE	Pending
Architecture Governance	docs/adr/, CODEOWNERS, tools/	ADRs, dependency policy, release standards, evidence	Architecture Review Board	Pending

Configuration-Driven Design and No-Hardcoding Standard

No-hardcoding means tenant-specific and operational behavior is resolved from validated configuration, not that all stable domain invariants become editable. State-machine invariants, security protocol values, and contract enums remain code-controlled; business preferences and thresholds are data/configuration-controlled.

Configuration Class	Source of Truth	Examples	Control Standard
Secrets	Enterprise secret manager / workload identity	DB credentials, Razorpay secrets, signing keys, provider tokens	Never committed, logged, returned by read APIs, or stored in plaintext
Deployment configuration	Typed environment schema	Ports, endpoints, timeouts, pool sizes, telemetry exporters	Fail-fast startup validation; environment-specific overlays
Tenant business settings	Versioned tenant-service records	Approval thresholds, languages, visibility default, receipt settings	Effective dates, optimistic locking, audit, cache invalidation event
Reference data	Versioned reference-data catalog	Event types, expense categories, units, document types	Stable code, localized labels, active dates, no destructive delete when referenced
Feature flags	Audited feature-flag service/table	Gateway rollout, new report, optional card mode	Owner, scope, expiry, percentage/cohort, kill switch, audit
Templates and localization	Versioned template catalog	Receipts, email, WhatsApp, Hindi/Marathi/English labels	Preview, approval, immutable version on generated document/message
Code invariants	Version-controlled source	Lifecycle states, permission protocol, event schema versions	ADR and compatibility review required to change

- Controllers, handlers, and repositories must not contain tenant names, account IDs, gateway keys, email addresses, receipt text, role display names, approval amounts, retry counts, URLs, or provider-specific magic constants.
- Every configurable key has a typed schema, owner, default, allowed range, sensitivity classification, and documentation. Unknown or invalid keys fail validation.
- Configuration changes publish config.changed.v1 events. Consumers invalidate caches by version, not by time-only TTL.
- Sensitive configuration requires step-up authentication, reason, masked before/after audit snapshots, and dual review where defined by policy.
- Feature flags must have an expiry/removal date. Permanent flags are converted into explicit configuration or removed.

Enterprise Testing and Quality Engineering Strategy

Mandatory Test Layers

Layer	Tooling	Scope	Environment	Release Gate
Unit	Jest	Pure domain rules, state machines, formulas, serializers, security helpers	In-process	All changed logic; deterministic and fast
Service integration	Jest/Supertest + Testcontainers	Real PostgreSQL, Redis, NATS, migrations, outbox/inbox, repository and HTTP/gRPC adapters	Ephemeral per pipeline	Mandatory for every service

Layer	Tooling	Scope	Environment	Release Gate
Contract	Pact plus OpenAPI/Protobuf/AsyncAPI compatibility	Consumer/provider expectations and backward compatibility	Provider verification pipeline	No breaking contract without new version
API E2E	Playwright APIRequestContext	Gateway-to-service workflows, auth, RBAC, tenant isolation, idempotency, webhooks	Ephemeral full stack	Mandatory smoke and critical finance journeys
Browser E2E	Playwright Chromium/Firefox/WebKit	User journeys once web frontend is connected; API post-condition validation	QA/staging and PR smoke	Critical journey pass; trace/video on failure
Performance	k6	Load, spike, soak, webhook bursts, report/export queues	Performance environment	SLO and capacity thresholds
Security	SAST, dependency/container scan, DAST, secret scan	OWASP risks, supply chain, images, exposed endpoints	PR and staging	Zero critical/high unaccepted findings
Resilience	Controlled fault/chaos tests	Timeouts, message redelivery, DLQ, database failover, pod loss	Staging	No duplicate financial side effects; recovery within SLO

Playwright Critical Journey Catalogue

- Organization registration -> Owner login -> role creation -> member invitation/acceptance.
- Donor self-registration and organization-assisted offline donor claim without duplicate identity.
- Donation submission -> authorized confirmation -> ledger posting -> receipt number allocation -> PDF/QR verification -> notification.
- Partial donation installments generate separate immutable receipts and reconcile to donation total.
- Expense draft -> voucher upload -> threshold policy snapshot -> distinct sequential approvals -> partial/full payment -> ledger reconciliation.
- Razorpay success webhook, duplicate webhook replay, invalid signature rejection, failed payment, and eligible duplicate-payment refund.
- Public transparency publication respects donor privacy and exposes only redacted voucher derivatives.
- Cross-tenant identifier substitution, export/file access, cache-key collision, and event-message tampering return 403/404 without data leakage.
- Event financial closure with outstanding liabilities, reopening reason, and preserved historical snapshots.

Quality Gates

- Overall changed-code coverage target: at least 80%; financial calculations, authorization, tenancy, idempotency, receipt numbering, ledger, payment signatures, and audit immutability: at least 90% branch coverage.
- Flaky tests are quarantined only with an owner, incident ticket, and removal deadline. Automatic retries must not convert a failing release gate into a pass without evidence.
- Each test run uses isolated tenants, databases/schemas, buckets/prefixes, queues, and provider stubs. Production data is prohibited in non-production testing.
- CI retains Playwright HTML reports, traces, screenshots/videos on failure, JUnit results, coverage, contract reports, migration evidence, performance summaries, and security scan outputs.
- Production deployment requires smoke tests after rollout and automated rollback when critical health, error-rate, or financial-consistency checks fail.

Container, Kubernetes, and DevSecOps Standards

- Each application has a multi-stage Dockerfile producing a minimal, pinned, non-root runtime image. Images run with a read-only root filesystem, dropped Linux capabilities, explicit resource limits, and no embedded secrets.
- Local development uses Docker Compose for PostgreSQL service databases, NATS JetStream, Redis, object-storage emulator, mail/SMS/WhatsApp stubs, and the selected services.
- Production uses managed Kubernetes with one Deployment per service/worker, ClusterIP services, Ingress/API Gateway, HorizontalPodAutoscaler, PodDisruptionBudget, topology spread/anti-affinity, readiness/liveness/startup probes, and graceful shutdown.
- Helm charts provide common templates and service-specific values. Environment promotion uses immutable image digests through development, QA, staging, and production.
- Database migrations run as controlled pre-deploy jobs using expand-and-contract changes. Destructive migrations require backup evidence, owner approval, and a tested rollback/forward-fix plan.

- NetworkPolicies default-deny east-west traffic. Service-to-service traffic uses mTLS/workload identity; database users are unique per service and least-privileged.
- CI generates an SBOM, scans dependencies and images, signs release images/attestations, and blocks unapproved critical vulnerabilities or leaked secrets.
- Production rollout uses canary or blue/green strategy for critical services. Automated rollback triggers on health, latency, error-rate, message-lag, or reconciliation thresholds.
- Kubernetes manifests, Helm values, policies, dashboards, alerts, and runbooks are version-controlled; manual production drift is prohibited.

Observability, SRE, and Operational Readiness

- OpenTelemetry instrumentation is mandatory for REST, gRPC, NATS, BullMQ, Prisma, Redis, and external provider calls. W3C trace context and correlationId propagate across every boundary.
- Logs are structured JSON and include timestamp, service, version, environment, traceId, spanId, requestId/messageId/jobId, organizationId hash or safe ID, actor ID, route/subject, outcome, duration, and error code. Secrets and sensitive payloads are redacted.
- Metrics include RED signals for APIs, queue/event lag, consumer redelivery, DLQ depth, DB pool saturation, cache behavior, PDF/export duration, webhook verification failures, receipt allocation conflicts, and ledger reconciliation mismatches.
- SLOs are defined per service and critical journey. Minimum production targets are 99.9% monthly availability for public/API services and 99.95% successful exactly-once business processing for accepted financial commands, excluding declared maintenance.
- Alerts must be actionable and linked to a runbook, owner, severity, dashboard, and escalation path. Paging is reserved for user-impacting or financial-integrity risks.
- Every service exposes /health/live, /health/ready, and /health/startup plus dependency-specific readiness checks. Readiness must fail before a pod accepts traffic when required dependencies are unavailable.
- Quarterly disaster-recovery, backup-restore, message-replay, key-rotation, and incident-response exercises are mandatory. Results and corrective actions are retained as release evidence.

Architecture, Security, and Delivery Governance

- Architecture Decision Records are mandatory for service boundaries, data ownership, contract changes, security mechanisms, financial invariants, new infrastructure, and exceptions to standards.
- An Architecture Review Board reviews cross-service changes; domain teams own internal implementation. CODEOWNERS and protected branches enforce approval requirements.
- Public APIs, event schemas, and Protobuf contracts use semantic versioning. Breaking changes require a new major/versioned route or subject and a published migration plan.
- Deprecated contracts remain supported for at least 90 days or two production releases, whichever is longer, unless an emergency security decision is approved and communicated.
- Threat modeling is required for authentication, payments, files, public publication, tenant isolation, ledger, audit, and administrative functions. Security abuse cases become automated tests where feasible.
- Release evidence includes approved PRs, test/coverage reports, contract verification, migration plan, SBOM and scan results, signed image digest, deployment diff, dashboards, alerts, runbook, and rollback result.
- Production access is just-in-time, least-privileged, MFA-protected, and audited. Direct database changes to financial or audit data are prohibited outside an approved break-glass procedure.

Module Specification: Authentication & Access Control

Overview & Metadata

Module Path	apps/identity-service/src/modules/auth/
Default Assignee/Role	Unassigned - Security Engineering
Initial Status	Pending
Core Responsibility	Mobile OTP and email/password authentication, JWT access/refresh sessions, lockout, and global guards.

1. Functional Requirements & Business Logic

1. Support verified mobile number plus OTP and verified email plus password. An active user must have at least one verified identity.
2. Hash passwords with Argon2id. Store only hashes of OTPs and refresh tokens; never log or persist raw credentials.
3. OTP challenges expire after five minutes, are single-use, allow at most five attempts, and are rate-limited by identity, IP, and device.
4. Issue 15-minute Access JWTs containing userId, platformRole, membershipId, organizationId, roleId, tokenVersion, and sessionId.
5. Issue rotated 30-day Refresh JWTs in Secure, HttpOnly, SameSite cookies. Reuse detection revokes the complete token family.
6. Lock password login for 15 minutes after five consecutive failures. Successful authentication resets the counter.
7. Use JwtAuthGuard, OrganizationContextGuard, MembershipGuard, and PermissionGuard on the server. Never trust client-side checks.
8. Revoke sessions immediately when a user is deactivated, membership is removed, role is changed, or organization is suspended.
9. Audit successful/failed login, logout, reset, token reuse, logout, and logout-all events without storing secrets.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
enum AuthProvider { MOBILE_OTP EMAIL_PASSWORD }
enum AuthSessionStatus { ACTIVE REVOKED EXPIRED COMPROMISED }
enum OtpPurpose { LOGIN VERIFY_MOBILE PASSWORD_RESET INVITATION_ACCEPTANCE }

model AuthIdentity {
  id          String      @id @default(uuid()) @db.Uuid
  userId      String      @db.Uuid
  provider     AuthProvider
  normalizedValue String  @db.VarChar(320)
  isVerified   Boolean    @default(false)
  passwordHash String?
  failedLoginCount Int     @default(0)
  lockedUntil  DateTime?
  lastAuthenticatedAt DateTime?
  createdAt    DateTime   @default(now())
  updatedAt    DateTime   @updatedAt
  user         User       @relation(fields: [userId], references: [id], onDelete: Cascade)
  @@unique([provider, normalizedValue])
  @@index([userId, isVerified])
}

model OtpChallenge {
  id          String      @id @default(uuid()) @db.Uuid
  userId      String?     @db.Uuid
  normalizedMobile String  @db.VarChar(20)
  purpose     OtpPurpose
  otpHash     String
  attemptCount Int         @default(0)
  maxAttempts Int         @default(5)
  expiresAt   DateTime
  consumedAt  DateTime?
  requestIpHash String?
  createdAt   DateTime   @default(now())
  user        User?      @relation(fields: [userId], references: [id], onDelete: Cascade)
  @@index([normalizedMobile, purpose, expiresAt])
}

model RefreshSession {
  id          String      @id @default(uuid()) @db.Uuid
  userId      String      @db.Uuid
  tokenFamilyId String    @db.Uuid
  tokenHash   String      @unique
  status      AuthSessionStatus @default(ACTIVE)
  tokenVersion Int        @default(1)
  userAgentHash String?
  ipAddressHash String?
  expiresAt   DateTime
  rotatedAt   DateTime?
  revokedAt   DateTime?
  revocationReason String?
  createdAt   DateTime   @default(now())
  user        User       @relation(fields: [userId], references: [id], onDelete: Cascade)
  @@index([userId, status])
  @@index([tokenFamilyId])
  @@index([expiresAt])
}
```

3. API Specifications

Method	Route	Purpose	Role Gate (RBAC)
POST	/api/v1/auth/otp/request	Create a rate-limited OTP challenge.	Public
POST	/api/v1/auth/otp/verify	Verify OTP and issue access/refresh sessions.	Public
POST	/api/v1/auth/login	Authenticate verified email and password.	Public
POST	/api/v1/auth/refresh	Rotate refresh token and issue access token.	Valid refresh session
POST	/api/v1/auth/logout	Revoke current session and clear cookie.	Authenticated
POST	/api/v1/auth/logout-all	Revoke all user sessions.	Authenticated
POST	/api/v1/auth/password/forgot	Create password-reset challenge.	Public
POST	/api/v1/auth/password/reset	Reset password and revoke sessions.	Valid reset challenge

4. Verification & Testing Tasks

- ☐ Mandatory enterprise gate: pass service integration with Testcontainers, consumer/provider contract compatibility, Playwright API E2E for externally reachable workflows, tenant-isolation negative tests, idempotency/redelivery tests, and observability assertions where applicable.

Unit & Cryptography Tests

- ☐ Verify Argon2id hashing, OTP hashing, JWT claims, expiry, issuer, audience, and signature rejection.
- ☐ Verify refresh-family rotation and reuse detection.

Integration Tests

- ☐ Lock exactly after configured failures; reject expired/consumed OTPs.
- ☐ Reject refresh for deactivated users and removed memberships.

RBAC & Security Checks

- ☐ Forge organization context and expect 403.
- ☐ Verify production cookie flags and secret-redacted logs.

Module Specification: Unified User Accounts

Overview & Metadata

Module Path	apps/identity-service/src/modules/users/
Default Assignee/Role	Unassigned - Core Backend
Initial Status	Pending
Core Responsibility	Canonical person identity, preferred language, account status, verified contacts, and safe profile administration.

1. Functional Requirements & Business Logic

- 10. Maintain one canonical user record referenced by donor profiles, authentication identities, organization memberships, and platform roles.
- 11. Support English, Marathi, and Hindi preferred languages with organization default and English fallback.
- 12. Require at least one verified mobile or email identity before protected financial actions.
- 13. Allow self-service updates to display name, language, and avatar. Mobile/email changes require re-verification.
- 14. Deactivation blocks login and protected actions immediately while preserving all historical references.
- 15. Platform Super Admin may deactivate/reactivate with a mandatory reason but cannot delete financial history.
- 16. Normalize mobile to E.164 and email to lowercase before uniqueness checks.
- 17. Never expose password hashes, OTPs, refresh hashes, security counters, or private organization data in profile responses.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
enum UserStatus { ACTIVE DEACTIVATED LOCKED MERGED }
enum PreferredLanguage { EN MR HI }
enum PlatformRole { USER SUPER_ADMIN }

model User {
  id                String          @id @default(uuid()) @db.Uuid
  displayName       String          @db.VarChar(150)
  preferredLanguage PreferredLanguage @default(EN)
  platformRole     PlatformRole    @default(USER)
  status           UserStatus      @default(ACTIVE)
  avatarDocumentId String?         @db.Uuid
  primaryMobile    String?         @unique @db.VarChar(20)
  primaryEmail     String?         @unique @db.VarChar(320)
  mobileVerifiedAt DateTime?
  emailVerifiedAt  DateTime?
  tokenVersion     Int             @default(1)
  deactivatedAt    DateTime?
  deactivationReason String?       @db.Text
  mergedIntoUserId String?         @db.Uuid
  createdAt        DateTime        @default(now())
  updatedAt        DateTime        @updatedAt
  mergedInto       User?            @relation("UserMerge", fields: [mergedIntoUserId], references: [id])
  mergedUsers      User[]           @relation("UserMerge")
  identities       AuthIdentity[]
  sessions         RefreshSession[]
  @@index([status, createdAt])
  @@index([platformRole, status])
}
```

3. API Specifications

Method	Route	Purpose	Role Gate (RBAC)
GET	/api/v1/users/me	Return authenticated user and active context.	Authenticated
PATCH	/api/v1/users/me	Update display name, language, and avatar.	Authenticated
POST	/api/v1/users/me/email/change	Begin verified email change.	Authenticated
POST	/api/v1/users/me/mobile/change	Begin verified mobile change.	Authenticated

Method	Route	Purpose	Role Gate (RBAC)
GET	/api/v1/platform/users/:id	Retrieve safe administrative profile.	Platform Super Admin
PATCH	/api/v1/platform/users/:id/status	Deactivate or reactivate with reason.	Platform Super Admin

4. Verification & Testing Tasks

- ☐ Mandatory enterprise gate: pass service integration with Testcontainers, consumer/provider contract compatibility, Playwright API E2E for externally reachable workflows, tenant-isolation negative tests, idempotency/redelivery tests, and observability assertions where applicable.

Unit Tests

- ☐ Validate contact normalization and language defaults.
- ☐ Verify safe serializers exclude sensitive fields.

Integration Tests

- ☐ Create mobile-only and email-only accounts.
- ☐ Reject duplicate normalized contact values with 409.
- ☐ Invalidate sessions after deactivation.

Security Checks

- ☐ Normal users cannot access platform administration.
- ☐ Profile PATCH cannot modify platformRole or verified timestamps.

Module Specification: Donor Profiles & Duplicate Merge

Overview & Metadata

Module Path	apps/donor-service/src/modules/donors/
Default Assignee/Role	Unassigned - Core Backend
Initial Status	Pending
Core Responsibility	Donor-specific profiles, organization-assisted offline creation, cross-organization donation history, and audited merging.

1. Functional Requirements & Business Logic

- 18. Every donation must reference an identified donor; anonymous donations are prohibited.
- 19. Donors may self-register. Authorized organization users and Platform Super Admin may create an unclaimed donor during offline donation entry.
- 20. Minimum donor data is full name plus mobile or email. A created profile can later be claimed after identity verification.
- 21. Before creation, search normalized mobile and email for exact candidates to prevent duplicate profiles.
- 22. A donor may donate to multiple organizations and events and view only their own private history.
- 23. Platform Super Admin may merge duplicates by selecting a survivor, re-pointing references transactionally, storing aliases, and deactivating the duplicate.
- 24. Merge requires a mandatory reason and fails while both accounts have conflicting active organization memberships.
- 25. Never combine or rewrite transaction values during merge; only change ownership references.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
// Service-owned Prisma schema. Cross-service references are scalar UUIDs; direct ORM joins and foreign keys across services are prohibited.
enum DonorProfileStatus { UNCLAIMED ACTIVE DEACTIVATED MERGED }

model DonorProfile {
  id          String      @id @default(uuid()) @db.Uuid
  userId      String      @unique @db.Uuid // external identity-service aggregate ID; no cross-service FK
  fullName    String      @db.VarChar(150)
  mobile      String?     @db.VarChar(20)
  email       String?     @db.VarChar(320)
  status      DonorProfileStatus @default(UNCLAIMED)
  createdByUserId String?  @db.Uuid
  createdByOrgId String?   @db.Uuid // external tenant-service aggregate ID; no cross-service FK
  claimedAt   DateTime?
  deactivatedAt DateTime?
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
  @@index([mobile])
  @@index([email])
  @@index([createdByOrgId, createdAt])
}

model DonorAlias {
  id          String      @id @default(uuid()) @db.Uuid
  survivingDonorId String  @db.Uuid
  mergedDonorId String    @unique @db.Uuid
  previousMobile String?
  previousEmail String?
  mergeLogId  String      @db.Uuid
  createdAt   DateTime     @default(now())
  survivingDonor DonorProfile @relation(fields: [survivingDonorId], references: [id])
  @@index([survivingDonorId])
}

model DonorMergeLog {
  id          String      @id @default(uuid()) @db.Uuid
```

```
survivingDonorId String @db.Uuid
mergedDonorId String @db.Uuid
performedByUserId String @db.Uuid
reason String @db.Text
referenceCounts Json
preMergeSnapshot Json
createdAt DateTime @default(now())
@@unique([survivingDonorId, mergedDonorId])
}
```

3. API Specifications

Method	Route	Purpose	Role Gate (RBAC)
POST	/api/v1/donors	Create or match an offline donor.	donor.create
GET	/api/v1/donors	Search organization-related donors.	donor.view
GET	/api/v1/donors/:id	Retrieve profile and scoped history.	donor.view
PATCH	/api/v1/donors/:id	Update permitted donor fields.	donor.update or self
GET	/api/v1/donors/me/history	View own cross-organization history.	Authenticated donor
POST	/api/v1/platform/donors/merge	Merge duplicate profiles transactionally.	Platform Super Admin

4. Verification & Testing Tasks

- ☐ Mandatory enterprise gate: pass service integration with Testcontainers, consumer/provider contract compatibility, Playwright API E2E for externally reachable workflows, tenant-isolation negative tests, idempotency/redelivery tests, and observability assertions where applicable.

Unit Tests

- ☐ Reject anonymous donations and invalid donor identities.
- ☐ Test candidate matching by mobile/email.

Integration Tests

- ☐ Claim an offline profile and retain donation history.
- ☐ Merge duplicates without changing monetary values.
- ☐ Reject merge on conflicting active memberships.

Privacy Checks

- ☐ Organizations cannot browse unrelated donor histories.
- ☐ Donors can view only their own private data.

Module Specification: Platform Administration

Overview & Metadata

Module Path	apps/platform-admin-service/src/modules/platform-admin/
Default Assignee/Role	Unassigned - Platform Operations
Initial Status	Pending
Core Responsibility	Platform-level organization and donor administration without access to tenant-private finance.

1. Functional Requirements & Business Logic

- 26. View organization identity, status, owner contact, registration metadata, and aggregate platform usage counts.
- 27. Platform Super Admin may suspend, reactivate, reject, require correction, or close an organization with a mandatory reason.
- 28. Organizations are active immediately after registration; platform approval is not required.
- 29. Rejected or correction-required organizations may update information and resubmit without losing history.
- 30. Platform Super Admin may create, deactivate, reactivate, and merge donors through the donor module.
- 31. Platform Super Admin cannot view donation amounts, expense details, ledgers, private vouchers, or tenant financial reports.
- 32. Platform Super Admin cannot confirm donations, approve expenses, alter ledger balances, or impersonate organization members.
- 33. Every platform action creates a platform-scope append-only audit entry.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
// Service-owned Prisma schema. Cross-service references are scalar UUIDs; direct ORM joins and foreign keys across services are prohibited.
enum PlatformCaseType {
  ORGANIZATION_CORRECTION ORGANIZATION_SUSPENSION
  ORGANIZATION_REJECTION ORGANIZATION_CLOSURE DONOR_ACCOUNT_REVIEW
}
enum PlatformCaseStatus { OPEN AWAITING_RESPONSE RESOLVED CLOSED }

model PlatformAdminCase {
  id          String      @id @default(uuid()) @db.Uuid
  caseType    PlatformCaseType
  status      PlatformCaseStatus @default(OPEN)
  organizationId String?   @db.Uuid // external tenant-service aggregate ID; no cross-service FK
  donorProfileId String?   @db.Uuid
  openedByUserId String    @db.Uuid
  assignedToUserId String?  @db.Uuid
  reason      String      @db.Text
  requestedActions Json?
  resolutionNotes String?  @db.Text
  openedAt    DateTime     @default(now())
  resolvedAt  DateTime?
  @@index([status, caseType, openedAt])
  @@index([organizationId])
  @@index([donorProfileId])
}
```

3. API Specifications

Method	Route	Purpose	Role Gate (RBAC)
GET	/api/v1/platform/organizations	List organizations and operational metadata.	Platform Super Admin
GET	/api/v1/platform/organizations/:id	View profile/compliance metadata.	Platform Super Admin

Method	Route	Purpose	Role Gate (RBAC)
PATCH	/api/v1/platform/organizations/:id/status	Change platform-controlled status.	Platform Super Admin
GET	/api/v1/platform/metrics	View non-financial usage aggregates.	Platform Super Admin
GET	/api/v1/platform/cases	List administrative cases.	Platform Super Admin
POST	/api/v1/platform/cases	Open a case.	Platform Super Admin
PATCH	/api/v1/platform/cases/:id	Assign or resolve a case.	Platform Super Admin

4. Verification & Testing Tasks

- ☐ Mandatory enterprise gate: pass service integration with Testcontainers, consumer/provider contract compatibility, Playwright API E2E for externally reachable workflows, tenant-isolation negative tests, idempotency/redelivery tests, and observability assertions where applicable.

Unit Tests

- ☐ Validate allowed status transitions and mandatory reasons.
- ☐ Verify financial-field redaction.

Integration Tests

- ☐ Suspend tenant and revoke member access while keeping receipt verification available.
- ☐ Resubmit rejected registration and audit the change.

Security Checks

- ☐ Organization users cannot call platform routes.
- ☐ Platform administrators cannot call tenant finance actions.

Module Specification: Organization Management

Overview & Metadata

Module Path	apps/tenant-service/src/modules/organizations/
Default Assignee/Role	Unassigned - Core Backend
Initial Status	Pending
Core Responsibility	Self-service organization registration, lifecycle, legal profile, ownership, and immutable organization code.

1. Functional Requirements & Business Logic

- 34. Anyone may register an organization and initial Owner. Registration creates an ACTIVE tenant in one transaction.
- 35. Require name, address, admin name, and at least one admin contact. Logo, registration number, PAN, UPI, bank details, and certificate are optional at registration.
- 36. Generate a globally unique organization code used in receipt numbers. It becomes immutable after the first receipt.
- 37. Encrypt bank details, PAN, Razorpay credentials, and other secrets. Read APIs return masked values.
- 38. Only the Owner may edit legal identity and banking information.
- 39. Lifecycle values are ACTIVE, CORRECTION_REQUIRED, SUSPENDED, REJECTED, and CLOSED.
- 40. Suspension or closure blocks new financial operations but never deletes transactions, receipts, reports, or audit records.
- 41. Every tenant-owned repository operation must include trusted organizationId.
- 42. An organization may run multiple active events but must have exactly one active Owner.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
// Service-owned Prisma schema. Cross-service references are scalar UUIDs; direct ORM joins and foreign keys across services are prohibited.
enum OrganizationStatus { ACTIVE CORRECTION_REQUIRED SUSPENDED REJECTED CLOSED }

model Organization {
  id          String      @id @default(uuid()) @db.Uuid
  code        String      @unique @db.VarChar(20)
  name        String      @db.VarChar(200)
  addressLine1 String      @db.VarChar(250)
  addressLine2 String?    @db.VarChar(250)
  city        String      @db.VarChar(100)
  state       String      @db.VarChar(100)
  postalCode   String      @db.VarChar(12)
  countryCode  String      @default("IN") @db.Char(2)
  status       OrganizationStatus @default(ACTIVE)
  ownerUserId  String      @db.Uuid // external identity-service aggregate ID; no cross-service FK
  logoDocumentId String?    @db.Uuid
  registrationNumber String? @db.VarChar(100)
  panEncrypted String?
  registrationDocumentId String? @db.Uuid
  primaryMobile String?    @db.VarChar(20)
  primaryEmail String?    @db.VarChar(320)
  bankAccountConfigured Boolean @default(false)
  upiConfigured Boolean    @default(false)
  correctionNotes String?  @db.Text
  statusReason String?    @db.Text
  activatedAt  DateTime    @default(now())
  suspendedAt  DateTime?
  closedAt     DateTime?
  createdAt    DateTime    @default(now())
  updatedAt    DateTime    @updatedAt
  @@index([status, createdAt])
  @@index([ownerUserId])
  @@index([name])
}
```

3. API Specifications

Method	Route	Purpose	Role Gate (RBAC)
POST	/api/v1/organizations/register	Register active organization and Owner.	Public
GET	/api/v1/organizations/current	Retrieve active organization profile.	Active member
PATCH	/api/v1/organizations/current	Update permitted profile/legal fields.	Owner
PATCH	/api/v1/organizations/current/banking	Configure bank and UPI details.	Owner
POST	/api/v1/organizations/current/resubmit	Resubmit corrected information.	Owner
POST	/api/v1/organizations/current/close	Close organization with reason.	Owner

4. Verification & Testing Tasks

- ☐ Mandatory enterprise gate: pass service integration with Testcontainers, consumer/provider contract compatibility, Playwright API E2E for externally reachable workflows, tenant-isolation negative tests, idempotency/redelivery tests, and observability assertions where applicable.

Unit Tests

- ☐ Test unique code generation under concurrency.
- ☐ Validate address and owner contact rules.

Integration Tests

- ☐ Registration atomically creates organization, Owner role, membership, and settings.
- ☐ Suspended/closed tenant cannot create transactions.
- ☐ Sensitive fields are encrypted and masked.

Tenant Checks

- ☐ Cross-tenant profile access returns 403.
- ☐ organizationId cannot be changed by PATCH.

Module Specification: Organization Memberships & Invitations

Overview & Metadata

Module Path	apps/tenant-service/src/modules/memberships/
Default Assignee/Role	Unassigned - Access Control
Initial Status	Pending
Core Responsibility	Invitations, direct account creation, one-active-organization restriction, status management, and ownership transfer.

1. Functional Requirements & Business Logic

- 43. Owner may invite by mobile/email, create an account directly, or add an existing donor as a member.
- 44. Invitation acceptance is configurable; invitations may be accepted, expired, or revoked.
- 45. Enforce one active organization membership per user across the platform.
- 46. Members may be temporarily deactivated/reactivated. Removal preserves historical references and allows later membership elsewhere.
- 47. Only the Owner can invite, create, deactivate, reactivate, remove, or re-role members.
- 48. Ownership transfer is atomic, requires reason, targets an active same-organization member, and leaves exactly one Owner.
- 49. The outgoing Owner receives a selected non-owner role, defaulting to President/Administrator.
- 50. Membership changes revoke or refresh authorization sessions immediately.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
// Service-owned Prisma schema. Cross-service references are scalar UUIDs; direct ORM joins and foreign keys across services are prohibited.
enum MembershipStatus { INVITED ACTIVE INACTIVE REMOVED EXPIRED REVOKED }
enum InvitationDeliveryMethod { MOBILE EMAIL DIRECT_CREATE }

model OrganizationMembership {
  id          String      @id @default(uuid()) @db.Uuid
  organizationId String    @db.Uuid
  userId      String      @db.Uuid // external identity-service aggregate ID; no cross-service FK
  roleId      String      @db.Uuid
  status      MembershipStatus @default(ACTIVE)
  isOwner     Boolean      @default(false)
  invitedByUserId String?   @db.Uuid
  activatedAt  DateTime?
  deactivatedAt DateTime?
  removedAt   DateTime?
  statusReason String?     @db.Text
  createdAt   DateTime      @default(now())
  updatedAt   DateTime      @updatedAt
  organization Organization @relation(fields: [organizationId], references: [id], onDelete: Restrict)
  role        OrganizationRole @relation(fields: [roleId], references: [id], onDelete: Restrict)
  @@unique([organizationId, userId])
  @@index([userId, status])
  @@index([organizationId, status, roleId])
}

model OrganizationInvitation {
  id          String      @id @default(uuid()) @db.Uuid
  organizationId String    @db.Uuid
  invitedByUserId String    @db.Uuid
  targetMobile String?     @db.VarChar(20)
  targetEmail String?      @db.VarChar(320)
  roleId      String      @db.Uuid
  deliveryMethod InvitationDeliveryMethod
  tokenHash   String?     @unique
  status      MembershipStatus @default(INVITED)
  expiresAt   DateTime?
  acceptedByUserId String?  @db.Uuid
  acceptedAt  DateTime?
```

```

createdAt    DateTime          @default(now())
organization  Organization      @relation(fields: [organizationId], references: [id], onDelete: Cascade)
@@index([organizationId, status])
@@index([targetMobile])
@@index([targetEmail])
}

```

3. API Specifications

Method	Route	Purpose	Role Gate (RBAC)
GET	/api/v1/memberships	List organization members.	Owner
POST	/api/v1/memberships/invitations	Invite by mobile or email.	Owner
POST	/api/v1/memberships/direct	Create or attach a member account.	Owner
POST	/api/v1/memberships/invitations/:token/accept	Accept invitation when required.	Invited user
PATCH	/api/v1/memberships/:id/status	Deactivate, reactivate, or remove.	Owner
PATCH	/api/v1/memberships/:id/role	Assign one role.	Owner
POST	/api/v1/memberships/transfer-ownership	Transfer ownership atomically.	Owner

4. Verification & Testing Tasks

- ❑ Mandatory enterprise gate: pass service integration with Testcontainers, consumer/provider contract compatibility, Playwright API E2E for externally reachable workflows, tenant-isolation negative tests, idempotency/redelivery tests, and observability assertions where applicable.

Constraint Tests

- ❑ Validate invitation expiry and token hashing.
- ❑ Validate one-role and protected Owner rules.

Integration Tests

- ❑ Reject second active organization membership.
- ❑ Transfer ownership with exactly one resulting Owner.
- ❑ Removal keeps historical approvals linked.

RBAC Checks

- ❑ Non-Owner cannot manage membership.
- ❑ Invitation acceptance must match verified target identity.

Module Specification: Custom Roles & Permissions

Overview & Metadata

Module Path	apps/tenant-service/src/modules/roles/
Default Assignee/Role	Unassigned - Access Control
Initial Status	Pending
Core Responsibility	Protected default roles, unlimited custom roles, permission catalog, and role-only server-side authorization.

1. Functional Requirements & Business Logic

- 51. Each active membership has exactly one role. Direct user-specific permissions are not supported in the MVP.
- 52. Create default roles: Owner, President, Secretary, Treasurer, Donation Collector, Expense Approver, Auditor, and Member.
- 53. Only the Owner may create, update, activate, deactivate, or delete roles.
- 54. Owner is protected: its permissions cannot be removed and the role cannot be deleted or deactivated.
- 55. A custom role assigned to active members cannot be deleted until all members are reassigned.
- 56. Maintain granular permission codes for organization, membership, role, event, donor, donation, receipt, in-kind, vendor, expense, approval, account, ledger, payment, report, public page, document, audit, and settings.
- 57. Every protected controller action declares permission metadata and PermissionGuard resolves role from trusted membership context.
- 58. Role changes are audited and apply to future actions without altering frozen approval evidence.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
model Permission {
  id          String    @id @default(uuid()) @db.Uuid
  code        String    @unique @db.VarChar(100)
  module      String    @db.VarChar(50)
  action      String    @db.VarChar(50)
  description String    @db.VarChar(250)
  isActive    Boolean   @default(true)
  createdAt   DateTime  @default(now())
}

model OrganizationRole {
  id              String    @id @default(uuid()) @db.Uuid
  organizationId  String    @db.Uuid
  name            String    @db.VarChar(100)
  description     String?   @db.VarChar(250)
  isSystem        Boolean   @default(false)
  isOwnerRole     Boolean   @default(false)
  isActive        Boolean   @default(true)
  createdByUserId String    @db.Uuid
  createdAt       DateTime  @default(now())
  updatedAt       DateTime  @updatedAt
  organization    Organization @relation(fields: [organizationId], references: [id], onDelete: Cascade)
  permissions     RolePermission[]
  memberships     OrganizationMembership[]
  @@unique([organizationId, name])
  @@index([organizationId, isActive])
}

model RolePermission {
  roleId          String    @db.Uuid
  permissionId    String    @db.Uuid
  grantedByUserId String    @db.Uuid
  grantedAt       DateTime  @default(now())
  role            OrganizationRole @relation(fields: [roleId], references: [id], onDelete: Cascade)
  permission      Permission  @relation(fields: [permissionId], references: [id], onDelete: Restrict)
  @@id([roleId, permissionId])
  @@index([permissionId])
}
```

3. API Specifications

Method	Route	Purpose	Role Gate (RBAC)
GET	/api/v1/roles	List organization roles.	Owner
POST	/api/v1/roles	Create custom role.	Owner
GET	/api/v1/roles/permissions	List permission catalog.	Owner
GET	/api/v1/roles/:id	Retrieve role and permissions.	Owner
PATCH	/api/v1/roles/:id	Update role and permission set.	Owner
DELETE	/api/v1/roles/:id	Delete unused custom role.	Owner

4. Verification & Testing Tasks

- ☐ Mandatory enterprise gate: pass service integration with Testcontainers, consumer/provider contract compatibility, Playwright API E2E for externally reachable workflows, tenant-isolation negative tests, idempotency/redelivery tests, and observability assertions where applicable.

Unit Tests

- ☐ Protect Owner role and validate unique role names.
- ☐ Validate permission-code uniqueness.

Integration Tests

- ☐ Role permission changes affect authorization immediately.
- ☐ Deleting assigned role returns 409.

RBAC Checks

- ☐ Every protected route has explicit permission metadata.
- ☐ Non-Owner cannot mutate roles.

Module Specification: Event Management

Overview & Metadata

Module Path	apps/event-service/src/modules/events/
Default Assignee/Role	Unassigned - Business Domain
Initial Status	Pending
Core Responsibility	Multiple event creation, lifecycle enforcement, financial closure, reopening, and immutable event code.

1. Functional Requirements & Business Logic

59. Support multiple simultaneous events per organization. Initial focus is Ganpati with configurable future event types.
60. Lifecycle is DRAFT -> ACTIVE -> COMPLETED -> FINANCIALLY_CLOSED -> ARCHIVED; direct jumps are rejected.
61. Only the Owner may activate, complete, financially close, reopen, or archive.
62. Donations may be collected before the event starts and recorded after it ends while the event remains ACTIVE or COMPLETED.
63. Financial closure blocks new donations and new expenses but allows payment of already-approved outstanding expenses.
64. An event may close with outstanding liabilities. Reports show available balance and projected balance separately.
65. Reopening FINANCIALLY_CLOSED -> ACTIVE requires a reason and audit record but no override token.
66. ARCHIVED is terminal and read-only in the MVP.
67. Generate an organization-unique event code. It becomes immutable after the first receipt allocation.
68. Freeze event identity fields into receipts and public snapshots.

2. Data Model Schema (PostgreSQL / Prisma ORM)

// Service-owned Prisma schema. Cross-service references are scalar UUIDs; direct ORM joins and foreign keys across services are prohibited.

```
enum EventStatus { DRAFT ACTIVE COMPLETED FINANCIALLY_CLOSED ARCHIVED }
```

```
model Event {
  id          String   @id @default(uuid()) @db.Uuid
  organizationId String @db.Uuid // external tenant-service aggregate ID; no cross-service FK
  eventTypeCode String  @db.VarChar(50)
  code        String   @db.VarChar(20)
  name        String   @db.VarChar(200)
  description  String?  @db.Text
  location     String?  @db.VarChar(250)
  bannerDocumentId String? @db.Uuid
  financialYearStart Int
  startDate    DateTime
  endDate      DateTime
  collectionStartDate DateTime?
  collectionEndDate DateTime?
  targetAmountPaise BigInt?
  openingBalancePaise BigInt @default(0)
  status        EventStatus @default(DRAFT)
  publicEnabled Boolean @default(false)
  receiptSequenceLocked Boolean @default(false)
  activatedByUserId String? @db.Uuid
  activatedAt    DateTime?
  completedAt    DateTime?
  financiallyClosedAt DateTime?
  reopenedAt     DateTime?
  reopenReason   String? @db.Text
  archivedAt     DateTime?
  createdByUserId String @db.Uuid
  createdAt      DateTime @default(now())
  updatedAt      DateTime @updatedAt
  @@unique([organizationId, code])
  @@index([organizationId, status, startDate])
}
```

```
@@index([organizationId, financialYearStart])
}
```

3. API Specifications

Method	Route	Purpose	Role Gate (RBAC)
GET	/api/v1/events	List events with filters.	event.view
POST	/api/v1/events	Create draft event.	event.create
GET	/api/v1/events/:id	Retrieve event and financial status.	event.view
PATCH	/api/v1/events/:id	Update mutable event fields.	event.update
POST	/api/v1/events/:id/activate	Activate event.	Owner
POST	/api/v1/events/:id/complete	Complete event.	Owner
POST	/api/v1/events/:id/financial-close	Financially close event.	Owner
POST	/api/v1/events/:id/reopen	Reopen with reason.	Owner
POST	/api/v1/events/:id/archive	Archive event.	Owner

4. Verification & Testing Tasks

- ☐ Mandatory enterprise gate: pass service integration with Testcontainers, consumer/provider contract compatibility, Playwright API E2E for externally reachable workflows, tenant-isolation negative tests, idempotency/redelivery tests, and observability assertions where applicable.

Lifecycle Tests

- ☐ Test all valid/invalid transitions.
- ☐ Block new finance after closure but allow outstanding expense payments.

Calculation Tests

- ☐ Verify available and projected balances.

RBAC & Audit

- ☐ Only Owner can manage lifecycle.
- ☐ Reopening requires reason and audit snapshots.

Module Specification: Monetary Donations & Installments

Overview & Metadata

Module Path	apps/contribution-service/src/modules/donations/
Default Assignee/Role	Unassigned - Finance Domain
Initial Status	Pending
Core Responsibility	Donation pledges, partial installments, payment verification, duplicate cancellation, confirmation, and receipt triggers.

1. Functional Requirements & Business Logic

- 69. Donors, authorized Collectors, Owner, and Platform Super Admin may initiate a donation for a target event.
- 70. Every donation references an identified donor; anonymous donations are rejected.
- 71. Support Monetary, Sponsorship, Advertisement, Membership, Government Grant, and Institutional Grant contribution types.
- 72. Support Cash, UPI, Bank Transfer, Cheque, Razorpay, and optional Card payment modes.
- 73. Lifecycle supports DRAFT, PENDING_PAYMENT, PENDING_VERIFICATION, PENDING_CHEQUE_CLEARANCE, PARTIALLY_RECEIVED, CONFIRMED, RECEIPT_ISSUED, FAILED, REJECTED, CANCELLED, and CHEQUE_BOUNCED.
- 74. Only organization users with donation.confirm may verify cash, manual UPI, bank transfer, cheque clearance, or manual card.
- 75. Cheque remains pending until cleared; bounced cheque creates no receipt and no positive ledger entry.
- 76. Every confirmed installment creates a separate immutable receipt.
- 77. Manual confirmed donations are non-refundable in the MVP. Duplicate records can be cancelled through approval.
- 78. Cancellation requires donation.cancel, a separate approver, mandatory reason, and audit snapshot.
- 79. Use idempotency keys and external-reference uniqueness. Confirmation, ledger posting, and receipt creation are atomic.
- 80. Store money as integer paise; confirmed total is the sum of confirmed installments.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
// Service-owned Prisma schema. Cross-service references are scalar UUIDs; direct ORM joins and foreign keys across services are prohibited.
enum ContributionType {
  MONETARY SPONSORSHIP ADVERTISEMENT MEMBERSHIP
  GOVERNMENT_GRANT INSTITUTIONAL_GRANT
}
enum DonationStatus {
  DRAFT PENDING_PAYMENT PENDING_VERIFICATION PENDING_CHEQUE_CLEARANCE
  PARTIALLY_RECEIVED CONFIRMED RECEIPT_ISSUED FAILED REJECTED
  CANCELLED CHEQUE_BOUNCED
}
enum PaymentMode { CASH UPI BANK_TRANSFER CHEQUE RAZORPAY CARD }
enum InstallmentStatus {
  PENDING PENDING_VERIFICATION PENDING_CHEQUE_CLEARANCE
  CONFIRMED REJECTED FAILED CANCELLED CHEQUE_BOUNCED
}

model Donation {
  id          String      @id @default(uuid()) @db.Uuid
  organizationId String    @db.Uuid // external tenant-service aggregate ID; no cross-service FK
  eventId     String      @db.Uuid // external event-service aggregate ID; no cross-service FK
  donorProfileId String    @db.Uuid // external donor-service aggregate ID; no cross-service FK
  contributionType ContributionType
  status      DonationStatus @default(DRAFT)
  pledgedAmountPaise BigInt?
  confirmedAmountPaise BigInt @default(0)
  openAmount  Boolean        @default(false)
  purpose     String?        @db.VarChar(250)
  publicVisibility PublicVisibility @default(NAME_AND_AMOUNT)
  donorPrivacySelected Boolean @default(false)
  createdByUserId String      @db.Uuid
```

```
idempotencyKey    String      @unique @db.VarChar(100)
cancelledByUserId  String?      @db.Uuid
cancellationApprovedByUserId String? @db.Uuid
cancellationReason String?      @db.Text
cancelledAt        DateTime?
createdAt          DateTime      @default(now())
updatedAt          DateTime      @updatedAt
installments       DonationInstallment[]
@@index([organizationId, eventId, status])
@@index([donorProfileId, createdAt])
}

model DonationInstallment {
  id          String      @id @default(uuid()) @db.Uuid
  donationId   String      @db.Uuid
  organizationId String      @db.Uuid // external tenant-service aggregate ID; no cross-service FK
  eventId      String      @db.Uuid // external event-service aggregate ID; no cross-service FK
  sequence     Int
  amountPaise   BigInt
  paymentMode    PaymentMode
  receivingAccountId String? @db.Uuid
  status         InstallmentStatus @default(PENDING)
  transactionReference String? @db.VarChar(150)
  chequeNumber   String?      @db.VarChar(50)
  chequeDate     DateTime?
  bankName       String?      @db.VarChar(150)
  externalPaymentId String?    @unique @db.VarChar(150)
  submittedByUserId String      @db.Uuid
  verifiedByUserId String?      @db.Uuid
  verifiedAt      DateTime?
  rejectionReason String?      @db.Text
  receiptId       String?      @unique @db.Uuid
  createdAt       DateTime      @default(now())
  updatedAt       DateTime      @updatedAt
  donation         Donation      @relation(fields: [donationId], references: [id], onDelete: Restrict)
  @@unique([donationId, sequence])
  @@index([organizationId, eventId, status])
  @@index([receivingAccountId, verifiedAt])
}
```

3. API Specifications

Method	Route	Purpose	Role Gate (RBAC)
POST	/api/v1/events/:eventId/donations	Create donation/initial installment.	Donor, donation.create, or Platform Super Admin
GET	/api/v1/events/:eventId/donations	List event donations.	donation.view
GET	/api/v1/donations/:id	Retrieve donation/installments.	donation.view or owning donor
POST	/api/v1/donations/:id/installments	Submit installment.	Donor or donation.create
POST	/api/v1/installments/:id/confirm	Verify and post receipt/ledger.	donation.confirm
POST	/api/v1/installments/:id/reject	Reject with reason.	donation.confirm
POST	/api/v1/installments/:id/cheque-clear	Confirm cheque clearance.	donation.confirm
POST	/api/v1/installments/:id/cheque-bounce	Record cheque bounce.	donation.confirm
POST	/api/v1/donations/:id/cancel	Cancel duplicate with separate approval.	donation.cancel

4. Verification & Testing Tasks

- ☐ Mandatory enterprise gate: pass service integration with Testcontainers, consumer/provider contract compatibility, Playwright API E2E for externally reachable workflows, tenant-isolation negative tests, idempotency/redelivery tests, and observability assertions where applicable.

Unit & Math Tests

- ☐ Confirmed total equals installment sum without floating point.
- ☐ Validate pledge caps and lifecycle transitions.

Transactional Tests

- ☐ Confirmation atomically creates receipt, ledger, and notification job.
- ☐ Concurrent/idempotent requests create one financial result.
- ☐ Test cheque clear/bounce.

RBAC & Privacy

- ☐ Platform Super Admin can create but not confirm.
- ☐ Creator cannot self-approve cancellation.
- ☐ Donors cannot view others' private donations.

Module Specification: Receipt Books & Sequential Numbering

Overview & Metadata

Module Path	apps/contribution-service/src/modules/receipt-books/
Default Assignee/Role	Unassigned - Finance Domain
Initial Status	Pending
Core Responsibility	Multiple event receipt books, optional collector assignment, atomic event-year numbering, and permanent closure.

1. Functional Requirements & Business Logic

- 81. Allow multiple active receipt books per event; books may be assigned to a Collector or shared.
- 82. All books for the same organization, event, and calendar year share one event-year sequence.
- 83. Public number format is PP-{ORG_CODE}-{EVENT_CODE}-{YEAR}-{SEQUENCE}; book code remains internal by default.
- 84. Sequence resets per event and year and starts at 1.
- 85. Allocate numbers atomically inside the receipt creation transaction.
- 86. Cancelled receipt numbers remain permanently consumed and are never reused.
- 87. Book status is OPEN or CLOSED. Closed books cannot reopen.
- 88. Only Owner or receipt-book.manage may create, assign, or close books.
- 89. Organization and event codes lock after the first receipt number is allocated.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
// Service-owned Prisma schema. Cross-service references are scalar UUIDs; direct ORM joins and foreign keys across services are prohibited.
enum ReceiptBookStatus { OPEN CLOSED }

model ReceiptBook {
  id          String      @id @default(uuid()) @db.Uuid
  organizationId String    @db.Uuid // external tenant-service aggregate ID; no cross-service FK
  eventId     String      @db.Uuid // external event-service aggregate ID; no cross-service FK
  code        String      @db.VarChar(20)
  displayName  String      @db.VarChar(100)
  assignedCollectorId String? @db.Uuid
  status       ReceiptBookStatus @default(OPEN)
  openedByUserId String    @db.Uuid
  openedAt     DateTime     @default(now())
  closedByUserId String?   @db.Uuid
  closedAt     DateTime?
  closeReason  String?     @db.Text
  @@unique([organizationId, eventId, code])
  @@index([organizationId, eventId, status])
}

model ReceiptCounter {
  id          String @id @default(uuid()) @db.Uuid
  organizationId String @db.Uuid // external tenant-service aggregate ID; no cross-service FK
  eventId     String @db.Uuid // external event-service aggregate ID; no cross-service FK
  calendarYear Int
  lastSequence BigInt @default(0)
  updatedAt   DateTime @updatedAt
  @@unique([organizationId, eventId, calendarYear])
}

model ReceiptNumberReservation {
  id          String @id @default(uuid()) @db.Uuid
  organizationId String @db.Uuid // external tenant-service aggregate ID; no cross-service FK
  eventId     String @db.Uuid // external event-service aggregate ID; no cross-service FK
  receiptBookId String @db.Uuid
  calendarYear Int
  sequence    BigInt
  receiptNumber String @unique @db.VarChar(80)
```

```
receiptId    String  @unique @db.Uuid
allocatedAt  DateTime @default(now())
@@unique([organizationId, eventId, calendarYear, sequence])
@@index([receiptBookId, allocatedAt])
}
```

3. API Specifications

Method	Route	Purpose	Role Gate (RBAC)
GET	/api/v1/events/:eventId/receipt-books	List receipt books.	receipt-book.view
POST	/api/v1/events/:eventId/receipt-books	Create receipt book.	receipt-book.manage
PATCH	/api/v1/receipt-books/:id/assignment	Assign/unassign Collector.	receipt-book.manage
POST	/api/v1/receipt-books/:id/close	Permanently close book.	receipt-book.manage
GET	/api/v1/receipt-books/:id/usage	View allocated range and usage.	receipt-book.view

4. Verification & Testing Tasks

- ☐ Mandatory enterprise gate: pass service integration with Testcontainers, consumer/provider contract compatibility, Playwright API E2E for externally reachable workflows, tenant-isolation negative tests, idempotency/redelivery tests, and observability assertions where applicable.

Concurrency Tests

- ☐ Run 100 concurrent allocations and assert unique committed values.
- ☐ Rollback leaves no orphan reservation.

Lifecycle Tests

- ☐ Closed book cannot allocate or reopen.
- ☐ Collector assignment restrictions apply when enabled.

Integrity Checks

- ☐ Cancelled number stays consumed.
- ☐ Historical number survives master-data edits.

Module Specification: Immutable Receipts, PDF & QR Verification

Overview & Metadata

Module Path	apps/contribution-service/src/modules/receipts/
Default Assignee/Role	Unassigned - Documents & Finance
Initial Status	Pending
Core Responsibility	Immutable multilingual receipts, PDF rendering, QR verification, formal cancellation, and replacement.

1. Functional Requirements & Business Logic

- 90. Generate a receipt for every confirmed monetary installment and verified in-kind contribution.
- 91. Freeze organization, event, donor, amount/value, payment reference, Collector, signature, languages, and numbering at issue time.
- 92. Receipts cannot be edited or deleted. Corrections use formal cancellation and optional replacement with a new number.
- 93. Cancellation requires receipt.cancel, separate approver, mandatory reason, original snapshot, and audit log.
- 94. Original PDF remains accessible; replacement and original reference each other.
- 95. QR verification safely shows ISSUED, CANCELLED, PARTIALLY_REFUNDED, or REFUNDED.
- 96. Render English, Marathi, and Hindi using versioned Handlebars templates and Puppeteer.
- 97. Financial receipt commits before asynchronous PDF rendering. Failed rendering can be manually retried.
- 98. Generate amount in words from integer paise; never fetch mutable master values after issue.
- 99. Email and WhatsApp share the immutable PDF and verification URL.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
// Service-owned Prisma schema. Cross-service references are scalar UUIDs; direct ORM joins and foreign keys across services are prohibited.
enum ReceiptStatus {
  GENERATING ISSUED CANCELLED PARTIALLY_REFUNDED REFUNDED RENDER_FAILED
}
enum ReceiptSourceType { DONATION_INSTALLMENT IN_KIND_CONTRIBUTION }

model Receipt {
  id String @id @default(uuid()) @db.Uuid
  organizationId String @db.Uuid // external tenant-service aggregate ID; no cross-service FK
  eventId String @db.Uuid // external event-service aggregate ID; no cross-service FK
  receiptBookId String @db.Uuid
  sourceType ReceiptSourceType
  donationInstallmentId String? @unique @db.Uuid
  inKindContributionId String? @unique @db.Uuid
  donorProfileId String @db.Uuid // external donor-service aggregate ID; no cross-service FK
  receiptNumber String @unique @db.VarChar(80)
  sequence BigInt
  calendarYear Int
  status ReceiptStatus @default(GENERATING)
  amountPaise BigInt?
  estimatedValuePaise BigInt?
  verificationTokenHash String @unique
  snapshot Json
  pdfDocumentId String? @unique @db.Uuid
  issuedByUserId String @db.Uuid
  issuedAt DateTime?
  cancelledByUserId String? @db.Uuid
  cancellationApprovedByUserId String? @db.Uuid
  cancellationReason String? @db.Text
  cancelledAt DateTime?
  replacesReceiptId String? @unique @db.Uuid
  replacedByReceiptId String? @unique @db.Uuid
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  @@index([organizationId, eventId, issuedAt])
}
```

```
@@index([donorProfileId, issuedAt])
@@index([status, createdAt])
}

model ReceiptRenderJob {
  id          String @id @default(uuid()) @db.Uuid
  receiptId   String @db.Uuid
  templateVersion String @db.VarChar(50)
  localeSet   String @db.VarChar(30)
  attemptCount Int @default(0)
  lastErrorCode String?
  lastErrorSafe String? @db.Text
  queuedAt    DateTime @default(now())
  completedAt DateTime?
  @@index([receiptId, queuedAt])
}
```

3. API Specifications

Method	Route	Purpose	Role Gate (RBAC)
GET	/api/v1/receipts/:id	Retrieve immutable receipt.	receipt.view or owning donor
GET	/api/v1/receipts/:id/pdf	Get signed PDF download URL.	receipt.view or owning donor
POST	/api/v1/receipts/:id/share	Share through email/WhatsApp.	receipt.share or owning donor
POST	/api/v1/receipts/:id/cancel	Cancel with separate approval.	receipt.cancel
POST	/api/v1/receipts/:id/replace	Issue replacement receipt.	receipt.replace
POST	/api/v1/receipts/:id/render-retry	Retry failed rendering.	receipt.manage
GET	/api/v1/public/receipts/verify/:token	Display safe QR verification.	Public

4. Verification & Testing Tasks

- ☐ Mandatory enterprise gate: pass service integration with Testcontainers, consumer/provider contract compatibility, Playwright API E2E for externally reachable workflows, tenant-isolation negative tests, idempotency/redelivery tests, and observability assertions where applicable.

Immutability Tests

- ☐ PATCH/PUT/DELETE issued receipt is rejected.
- ☐ Frozen ORM fields cannot be updated.

PDF & Localization

- ☐ Validate amount-in-words and glyphs in all languages.
- ☐ Render representative receipts without clipping.

Cancellation Tests

- ☐ Original PDF remains and number is not reused.
- ☐ QR and cross-references update correctly.

Security Checks

- ☐ Public verification excludes contacts/internal IDs.
- ☐ Signed URLs expire.

Module Specification: In-Kind Contributions

Overview & Metadata

Module Path	apps/contribution-service/src/modules/in-kind-contributions/
Default Assignee/Role	Unassigned - Business Domain
Initial Status	Pending
Core Responsibility	Non-cash contribution intake, optional valuation, receipt issuance, storage/use notes, and separate reporting.

1. Functional Requirements & Business Logic

- 100.Support goods, materials, food, equipment, services, and other non-cash contributions from identified donors.
- 101.Capture item/service name, category, description, quantity, unit, receipt date, receiver, storage, usage, notes, and documents.
- 102.Organization setting determines whether valuation is required and whether verified value contributes to contribution statistics.
- 103.Only in-kind.verify may verify estimated/verified values, stored as integer paise.
- 104.In-kind value never increases any cash, bank, UPI, gateway, card, or available financial balance.
- 105.Lifecycle is DRAFT -> SUBMITTED -> RECEIVED -> VALUE_PENDING -> VERIFIED -> RECEIPT_ISSUED, with REJECTED and RETURNED alternatives.
- 106.If valuation is optional and absent, contribution can be quantity-only.
- 107.Returning a received contribution preserves the issued receipt and records return status/reason.
- 108 Freeze contribution data into the receipt.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
// Service-owned Prisma schema. Cross-service references are scalar UUIDs; direct ORM joins and foreign keys across services are prohibited.
enum InKindStatus {
  DRAFT SUBMITTED RECEIVED VALUE_PENDING VERIFIED RECEIPT_ISSUED REJECTED RETURNED
}

model InKindContribution {
  id          String    @id @default(uuid()) @db.Uuid
  organizationId String  @db.Uuid // external tenant-service aggregate ID; no cross-service FK
  eventId      String    @db.Uuid // external event-service aggregate ID; no cross-service FK
  donorProfileId String  @db.Uuid // external donor-service aggregate ID; no cross-service FK
  status       InKindStatus @default(DRAFT)
  itemName     String    @db.VarChar(200)
  category     String    @db.VarChar(100)
  description   String?   @db.Text
  quantity     Decimal   @db.Decimal(18,3)
  unit         String    @db.VarChar(50)
  estimatedValuePaise BigInt?
  verifiedValuePaise BigInt?
  valueIncludedInTotals Boolean @default(false)
  receivedByUserId String? @db.Uuid
  receivedAt    DateTime?
  verifiedByUserId String? @db.Uuid
  verifiedAt    DateTime?
  storageLocation String? @db.VarChar(200)
  usagePurpose  String?   @db.VarChar(250)
  notes        String?   @db.Text
  receiptId     String?   @unique @db.Uuid
  returnedAt    DateTime?
  returnReason  String?   @db.Text
  createdByUserId String  @db.Uuid
  createdAt     DateTime  @default(now())
  updatedAt     DateTime  @updatedAt
  attachments   InKindAttachment[]
  @@index([organizationId, eventId, status])
}
```

```

model InKindAttachment {
  contributionId String @db.Uuid
  documentId String @db.Uuid // external document-service aggregate ID; no cross-service FK
  sortOrder Int @default(0)
  contribution InKindContribution @relation(fields: [contributionId], references: [id], onDelete: Cascade)
  @@id([contributionId, documentId])
}

```

3. API Specifications

Method	Route	Purpose	Role Gate (RBAC)
POST	/api/v1/events/:eventId/in-kind-contributions	Create contribution.	Donor or in-kind.create
GET	/api/v1/events/:eventId/in-kind-contributions	List contributions.	in-kind.view
GET	/api/v1/in-kind-contributions/:id	Retrieve details.	in-kind.view or owning donor
PATCH	/api/v1/in-kind-contributions/:id	Edit before receipt issue.	in-kind.update
POST	/api/v1/in-kind-contributions/:id/receive	Mark received.	in-kind.receive
POST	/api/v1/in-kind-contributions/:id/verify	Verify and trigger receipt.	in-kind.verify
POST	/api/v1/in-kind-contributions/:id/return	Record return.	in-kind.manage

4. Verification & Testing Tasks

- ☐ Mandatory enterprise gate: pass service integration with Testcontainers, consumer/provider contract compatibility, Playwright API E2E for externally reachable workflows, tenant-isolation negative tests, idempotency/redelivery tests, and observability assertions where applicable.

Unit Tests

- ☐ Validate quantity precision and valuation settings.
- ☐ Ensure values never affect financial balances.

Integration Tests

- ☐ Verification creates receipt without monetary ledger credit.
- ☐ Return preserves historical receipt.

RBAC & Documents

- ☐ Only authorized users can receive/verify.
- ☐ Enforce five-attachment and private-access rules.

Module Specification: Vendor Management

Overview & Metadata

Module Path	apps/finance-service/src/modules/vendors/
Default Assignee/Role	Unassigned - Finance Domain
Initial Status	Pending
Core Responsibility	Organization-scoped vendor records required for expenses, with sensitive data protection and frozen snapshots.

1. Functional Requirements & Business Logic

- 109. Every expense must reference a vendor; vendorless expenses are rejected.
- 110. Capture name, contact, address, optional tax IDs, bank details, and active status.
- 111. Only vendor.manage may create or update vendors. Referenced vendors use soft deactivation instead of deletion.
- 112. Deduplicate by normalized name, mobile, tax ID fingerprint, and bank fingerprint within the organization.
- 113. Approved expenses freeze a vendor snapshot so later edits do not change history.
- 114. Encrypt bank and tax identifiers and return masked values.
- 115. Platform Super Admin cannot view private vendor details.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
// Service-owned Prisma schema. Cross-service references are scalar UUIDs; direct ORM joins and foreign keys across services are prohibited.
enum VendorStatus { ACTIVE INACTIVE }

model Vendor {
  id          String    @id @default(uuid()) @db.Uuid
  organizationId String  @db.Uuid // external tenant-service aggregate ID; no cross-service FK
  name        String    @db.VarChar(200)
  contactPerson String?  @db.VarChar(150)
  mobile       String?   @db.VarChar(20)
  email        String?   @db.VarChar(320)
  address      String?   @db.Text
  gstinEncrypted String?
  panEncrypted String?
  bankAccountEncrypted String?
  bankIfscEncrypted String?
  status       VendorStatus @default(ACTIVE)
  createdByUserId String   @db.Uuid
  deactivatedByUserId String? @db.Uuid
  deactivatedAt DateTime?
  createdAt    DateTime   @default(now())
  updatedAt    DateTime   @updatedAt
  @@unique([organizationId, name])
  @@index([organizationId, status])
  @@index([organizationId, mobile])
}
```

3. API Specifications

Method	Route	Purpose	Role Gate (RBAC)
GET	/api/v1/vendors	List/search vendors.	vendor.view
POST	/api/v1/vendors	Create vendor.	vendor.manage
GET	/api/v1/vendors/:id	Retrieve masked details.	vendor.view
PATCH	/api/v1/vendors/:id	Update active vendor.	vendor.manage
POST	/api/v1/vendors/:id/deactivate	Deactivate vendor.	vendor.manage
POST	/api/v1/vendors/:id/reactivate	Reactivate vendor.	vendor.manage

4. Verification & Testing Tasks

- ☐ Mandatory enterprise gate: pass service integration with Testcontainers, consumer/provider contract compatibility, Playwright API E2E for externally reachable workflows, tenant-isolation negative tests, idempotency/redelivery tests, and observability assertions where applicable.

Unit Tests

- ☐ Validate tenant uniqueness and field masking.

Integration Tests

- ☐ Approved expense keeps vendor snapshot after master edit.
- ☐ Historical expense remains readable after deactivation.

Security Checks

- ☐ Cross-organization access is denied.
- ☐ Platform Super Admin cannot access private vendor data.

Module Specification: Expense Approval Policies

Overview & Metadata

Module Path	apps/finance-service/src/modules/approval-policies/
Default Assignee/Role	Unassigned - Finance Architecture
Initial Status	Pending
Core Responsibility	Configurable expense thresholds, sequential distinct approvals, immutable versions, and submission-time snapshots.

1. Functional Requirements & Business Logic

- 116.Approval thresholds are mandatory and configurable per organization.
- 117.Default bands: up to INR 5,000 one approval; INR 5,001-25,000 two distinct approvals; above INR 25,000 two distinct approvals including Owner or Treasurer.
- 118.Approvals are sequential and later steps cannot be completed early.
- 119.Submitter cannot approve; one user cannot satisfy multiple levels; Owner cannot bypass.
- 120.Emergency override is excluded from the MVP.
- 121.Activated policy versions are immutable; changes create a new version and effective period.
- 122.Expense submission freezes the active policy band, required approvals, and role requirements.
- 123.Later policy changes do not alter submitted or approved expenses.
- 124.Only Owner may manage policies; Auditor may read versions.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
// Service-owned Prisma schema. Cross-service references are scalar UUIDs; direct ORM joins and foreign keys across services are prohibited.
model ExpenseApprovalPolicy {
  id          String  @id @default(uuid()) @db.Uuid
  organizationId String @db.Uuid // external tenant-service aggregate ID; no cross-service FK
  version     Int
  name        String  @db.VarChar(120)
  isActive    Boolean @default(false)
  effectiveFrom DateTime
  effectiveTo  DateTime?
  createdByUserId String @db.Uuid
  activatedByUserId String? @db.Uuid
  createdAt   DateTime @default(now())
  bands       ExpenseApprovalPolicyBand[]
  @@unique([organizationId, version])
  @@index([organizationId, isActive, effectiveFrom])
}

model ExpenseApprovalPolicyBand {
  id          String  @id @default(uuid()) @db.Uuid
  policyId    String  @db.Uuid
  minAmountPaise BigInt
  maxAmountPaise BigInt?
  requiredApprovals Int
  requiredRoleNames Json
  requireOwnerOrTreasurer Boolean @default(false)
  sequenceDefinition Json
  policy      ExpenseApprovalPolicy @relation(fields: [policyId], references: [id], onDelete: Cascade)
  @@index([policyId, minAmountPaise])
}

model ExpenseApprovalPolicySnapshot {
  id          String  @id @default(uuid()) @db.Uuid
  expenseId   String  @unique @db.Uuid
  sourcePolicyId String @db.Uuid
  sourcePolicyVersion Int
  evaluatedAmountPaise BigInt
  requiredApprovals Int
  steps       Json
}
```

```

snapshot      Json
createdAt     DateTime @default(now())
@@index([sourcePolicyId, sourcePolicyVersion])
}

```

3. API Specifications

Method	Route	Purpose	Role Gate (RBAC)
GET	/api/v1/approval-policies	List policy versions.	Owner or audit.view
POST	/api/v1/approval-policies	Create draft version.	Owner
PATCH	/api/v1/approval-policies/:id	Edit inactive draft.	Owner
POST	/api/v1/approval-policies/:id/activate	Activate version.	Owner
GET	/api/v1/approval-policies/evaluate	Preview approval requirements.	expense.create

4. Verification & Testing Tasks

- ☐ Mandatory enterprise gate: pass service integration with Testcontainers, consumer/provider contract compatibility, Playwright API E2E for externally reachable workflows, tenant-isolation negative tests, idempotency/redelivery tests, and observability assertions where applicable.

Boundary Tests

- ☐ Test INR 5,000, 5,000.01, 25,000, and 25,000.01 in paise.
- ☐ Reject overlapping/gapped bands.

Snapshot Tests

- ☐ New policy does not alter an already-submitted expense.

RBAC Checks

- ☐ Only Owner mutates policies.
- ☐ No emergency bypass endpoint exists.

Module Specification: Expense Management & Payments

Overview & Metadata

Module Path	apps/finance-service/src/modules/expenses/
Default Assignee/Role	Unassigned - Finance Domain
Initial Status	Pending
Core Responsibility	Vendor-backed expenses, mandatory vouchers, sequential approvals, partial payments, cancellation, and immutable approved records.

1. Functional Requirements & Business Logic

- 125.Permissions independently control expense.create, submit, approve, reject, pay, cancel, view, and export.
- 126.Every expense references event and vendor and includes category, description, date, base amount, tax amount, and at least one bill or signed internal voucher.
- 127.GrossAmountPaise = baseAmountPaise + taxAmountPaise. OutstandingAmountPaise = approvedAmountPaise - paidAmountPaise.
- 128.Lifecycle is DRAFT -> SUBMITTED -> UNDER_APPROVAL -> APPROVED -> PARTIALLY_PAID -> PAID, with REJECTED and CANCELLATION_PENDING -> CANCELLED.
- 129.Submission freezes business fields and policy snapshot; approved expenses are immutable.
- 130.Submitter cannot approve or record payment. Approval users must be distinct.
- 131.Rejection and cancellation require reasons. Approved unpaid cancellation requires a separate authorized approver.
- 132.Partial payments are allowed; cumulative payment cannot exceed approved amount.
- 133.Paid expenses cannot be edited or reversed. Corrections use a linked compensating credit/adjustment.
- 134.After event closure, existing approved outstanding expenses can still be paid; new expenses are blocked.
- 135.Gateway fees are system-generated expenses with dedicated category/vendor snapshot.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
// Service-owned Prisma schema. Cross-service references are scalar UUIDs; direct ORM joins and foreign keys across services are prohibited.
enum ExpenseStatus {
  DRAFT SUBMITTED UNDER_APPROVAL APPROVED PARTIALLY_PAID PAID
  REJECTED CANCELLATION_PENDING CANCELLED
}
enum ApprovalDecision { PENDING APPROVED REJECTED }
enum ExpensePaymentStatus { PENDING CONFIRMED FAILED CANCELLED }

model Expense {
  id          String      @id @default(uuid()) @db.Uuid
  organizationId String    @db.Uuid // external tenant-service aggregate ID; no cross-service FK
  eventId     String      @db.Uuid // external event-service aggregate ID; no cross-service FK
  vendorId    String      @db.Uuid
  status      ExpenseStatus @default(DRAFT)
  categoryCode String      @db.VarChar(80)
  description  String      @db.Text
  expenseDate  DateTime
  baseAmountPaise BigInt
  taxAmountPaise BigInt @default(0)
  requestedAmountPaise BigInt
  approvedAmountPaise BigInt?
  paidAmountPaise BigInt @default(0)
  outstandingAmountPaise BigInt?
  vendorSnapshot Json?
  policySnapshotId String? @unique @db.Uuid
  submittedByUserId String? @db.Uuid
  submittedAt    DateTime?
  rejectedByUserId String? @db.Uuid
  rejectionReason String? @db.Text
  rejectedAt     DateTime?
```

```

cancellationRequestedByUserId String? @db.Uuid
cancellationApprovedByUserId String? @db.Uuid
cancellationReason String? @db.Text
cancelledAt DateTime?
createdByUserId String @db.Uuid
createdAt DateTime @default(now())
updatedAt DateTime @updatedAt
vendor Vendor @relation(fields: [vendorId], references: [id], onDelete: Restrict)
approvals ExpenseApproval[]
payments ExpensePayment[]
attachments ExpenseAttachment[]
@@index([organizationId, eventId, status])
@@index([vendorId, expenseDate])
}

model ExpenseApproval {
  id String @id @default(uuid()) @db.Uuid
  expenseId String @db.Uuid
  stepNumber Int
  requiredRoleNames Json
  decision ApprovalDecision @default(PENDING)
  decidedByUserId String? @db.Uuid
  reason String? @db.Text
  decidedAt DateTime?
  createdAt DateTime @default(now())
  expense Expense @relation(fields: [expenseId], references: [id], onDelete: Restrict)
  @@unique([expenseId, stepNumber])
  @@index([decision, createdAt])
}

model ExpensePayment {
  id String @id @default(uuid()) @db.Uuid
  expenseId String @db.Uuid
  accountId String @db.Uuid
  amountPaise BigInt
  paymentMode PaymentMode
  transactionReference String? @db.VarChar(150)
  status ExpensePaymentStatus @default(PENDING)
  paidByUserId String @db.Uuid
  confirmedAt DateTime?
  ledgerTransactionId String? @unique @db.Uuid
  createdAt DateTime @default(now())
  expense Expense @relation(fields: [expenseId], references: [id], onDelete: Restrict)
  @@index([expenseId, status])
  @@index([accountId, confirmedAt])
}

model ExpenseAttachment {
  expenseId String @db.Uuid
  documentId String @db.Uuid // external document-service aggregate ID; no cross-service FK
  isVoucher Boolean @default(true)
  sortOrder Int @default(0)
  expense Expense @relation(fields: [expenseId], references: [id], onDelete: Restrict)
  @@id([expenseId, documentId])
}

```

3. API Specifications

Method	Route	Purpose	Role Gate (RBAC)
GET	/api/v1/events/:eventId/expenses	List event expenses.	expense.view
POST	/api/v1/events/:eventId/expenses	Create draft expense.	expense.create
GET	/api/v1/expenses/:id	Retrieve expense/workflow.	expense.view
PATCH	/api/v1/expenses/:id	Edit DRAFT only.	expense.update
POST	/api/v1/expenses/:id/submit	Freeze and create approvals.	expense.submit

Method	Route	Purpose	Role Gate (RBAC)
POST	/api/v1/expenses/:id/approvals/:step/approve	Approve current step.	expense.approve
POST	/api/v1/expenses/:id/approvals/:step/reject	Reject with reason.	expense.reject
POST	/api/v1/expenses/:id/payments	Record partial/final payment.	expense.pay
POST	/api/v1/expenses/:id/cancellation-request	Request approved-unpaid cancellation.	expense.cancel
POST	/api/v1/expenses/:id/cancellation-approve	Approve cancellation separately.	expense.cancel

4. Verification & Testing Tasks

- ☐ Mandatory enterprise gate: pass service integration with Testcontainers, consumer/provider contract compatibility, Playwright API E2E for externally reachable workflows, tenant-isolation negative tests, idempotency/redelivery tests, and observability assertions where applicable.

Unit & Math Tests

- ☐ Validate gross/paid/outstanding values in paise.
- ☐ Reject overpayment and non-positive payments.

Workflow Tests

- ☐ Enforce sequential distinct approvals.
- ☐ Reject field edits after submission.
- ☐ Verify partial/final payment states.

Integration Tests

- ☐ Payment atomically posts ledger and totals.
- ☐ Closed event permits only existing approved payments.

RBAC & Documents

- ☐ Submitter cannot approve/pay.
- ☐ Submission requires a private voucher.

Module Specification: Financial Accounts

Overview & Metadata

Module Path	apps/finance-service/src/modules/accounts/
Default Assignee/Role	Unassigned - Finance Architecture
Initial Status	Pending
Core Responsibility	Organization-level Cash, Bank, UPI, Cheque, Razorpay, and Card accounts with ledger-backed balances and transfers.

1. Functional Requirements & Business Logic

- 136.Allow at most one account per type per organization: CASH, BANK, UPI, CHEQUE_CLEARING, RAZORPAY_SETTLEMENT, CARD_SETTLEMENT.
- 137.Accounts are organization-level; event attribution occurs on source and ledger records.
- 138.Donations select the receiving account and expense payments select the paying account.
- 139.Opening balance is allowed only through an immutable opening ledger transaction.
- 140.Current balance is derived from posted ledger entries; direct balance editing is prohibited.
- 141.Internal transfers create equal debit/credit entries and do not count as donations or expenses.
- 142.Deactivate only when no pending payment references the account.
- 143.Encrypt sensitive identifiers and return masked display values.
- 144.Separate permissions control account.view, account.manage, and account.transfer.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
// Service-owned Prisma schema. Cross-service references are scalar UUIDs; direct ORM joins and foreign keys across services are prohibited.
enum FinancialAccountType {
  CASH BANK UPI CHEQUE_CLEARING RAZORPAY_SETTLEMENT CARD_SETTLEMENT
}
enum FinancialAccountStatus { ACTIVE INACTIVE }
enum TransferStatus { PENDING POSTED FAILED CANCELLED }

model FinancialAccount {
  id          String          @id @default(uuid()) @db.Uuid
  organizationId String        @db.Uuid // external tenant-service aggregate ID; no cross-service FK
  type        FinancialAccountType
  displayName  String          @db.VarChar(120)
  status       FinancialAccountStatus @default(ACTIVE)
  maskedIdentifier String?      @db.VarChar(100)
  encryptedConfiguration String?
  openingBalancePaise BigInt      @default(0)
  openingLedgerTransactionId String? @unique @db.Uuid
  createdByUserId String         @db.Uuid
  deactivatedByUserId String?     @db.Uuid
  deactivatedAt DateTime?
  createdAt    DateTime          @default(now())
  updatedAt    DateTime          @updatedAt
  @@unique([organizationId, type])
  @@index([organizationId, status])
}

model InternalTransfer {
  id          String          @id @default(uuid()) @db.Uuid
  organizationId String        @db.Uuid // external tenant-service aggregate ID; no cross-service FK
  eventId      String?         @db.Uuid
  sourceAccountId String        @db.Uuid
  destinationAccountId String    @db.Uuid
  amountPaise  BigInt
  status       TransferStatus @default(PENDING)
  reference    String?         @db.VarChar(150)
  description  String?         @db.VarChar(250)
  initiatedByUserId String      @db.Uuid
  ledgerTransactionId String?   @unique @db.Uuid
}
```

```

    postedAt      DateTime?
    createdAt      DateTime    @default(now())
    @@index([organizationId, createdAt])
    @@index([sourceAccountId, status])
    @@index([destinationAccountId, status])
  }

```

3. API Specifications

Method	Route	Purpose	Role Gate (RBAC)
GET	/api/v1/accounts	List masked accounts/balances.	account.view
POST	/api/v1/accounts	Create account and opening entry.	account.manage
PATCH	/api/v1/accounts/:id	Update metadata/configuration.	account.manage
POST	/api/v1/accounts/:id/deactivate	Deactivate unused account.	account.manage
POST	/api/v1/accounts/transfers	Post internal transfer.	account.transfer
GET	/api/v1/accounts/transfers	List transfers.	account.view

4. Verification & Testing Tasks

- ☐ Mandatory enterprise gate: pass service integration with Testcontainers, consumer/provider contract compatibility, Playwright API E2E for externally reachable workflows, tenant-isolation negative tests, idempotency/redelivery tests, and observability assertions where applicable.

Constraint Tests

- ☐ One account per type; source and destination differ.
- ☐ Positive paise values only.

Transactional Tests

- ☐ Transfer posts balanced entries atomically.
- ☐ Failure cannot leave partial transfer.

Security Checks

- ☐ Sensitive data is masked/encrypted.
- ☐ No API can patch current balance.

Module Specification: Immutable Financial Ledger

Overview & Metadata

Module Path	apps/finance-service/src/modules/ledger/
Default Assignee/Role	Unassigned - Finance Architecture
Initial Status	Pending
Core Responsibility	Append-only double-entry source of truth for all financial movements and reconciliation.

1. Functional Requirements & Business Logic

- 145. Every confirmed donation, expense payment, gateway fee, eligible refund, opening balance, and transfer creates a ledger transaction.
- 146. POSTED ledger transactions and entries are append-only with no update/delete API.
- 147. Each transaction must balance: total debits equal total credits.
- 148. Donation debits the receiving asset and credits donation income; expense debits expense and credits paying asset.
- 149. Internal transfers affect only asset accounts and do not affect donation/expense totals.
- 150. Refunds and corrections create linked compensating entries instead of editing originals.
- 151. All entries include organizationId and optional eventId; reports query posted data only.
- 152. Financial workflows use database transactions and idempotency keys.
- 153. Run reconciliation between source records and ledger and restrict results to finance/audit roles.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
enum LedgerTransactionType {
  OPENING_BALANCE DONATION_RECEIPT EXPENSE_PAYMENT GATEWAY_FEE
  REFUND INTERNAL_TRANSFER COMPENSATING_ADJUSTMENT
}
enum LedgerTransactionStatus { PENDING POSTED FAILED }
enum LedgerEntrySide { DEBIT CREDIT }
enum LedgerAccountClass { ASSET INCOME EXPENSE LIABILITY EQUITY }

model LedgerTransaction {
  id String @id @default(uuid()) @db.Uuid
  organizationId String @db.Uuid
  eventId String? @db.Uuid
  type LedgerTransactionType
  status LedgerTransactionStatus @default(PENDING)
  sourceType String @db.VarChar(80)
  sourceId String @db.Uuid
  idempotencyKey String @unique @db.VarChar(120)
  description String? @db.VarChar(250)
  postedByUserId String? @db.Uuid
  postedAt DateTime?
  createdAt DateTime @default(now())
  entries LedgerEntry[]
  @@unique([organizationId, sourceType, sourceId, type])
  @@index([organizationId, eventId, postedAt])
  @@index([status, createdAt])
}

model LedgerEntry {
  id String @id @default(uuid()) @db.Uuid
  ledgerTransactionId String @db.Uuid
  organizationId String @db.Uuid
  eventId String? @db.Uuid
  financialAccountId String? @db.Uuid
  accountCode String @db.VarChar(80)
  accountClass LedgerAccountClass
  side LedgerEntrySide
  amountPaise BigInt
  createdAt DateTime @default(now())
  transaction LedgerTransaction @relation(fields: [ledgerTransactionId], references: [id], onDelete: Restrict)
  @@index([organizationId, eventId, createdAt])
  @@index([financialAccountId, createdAt])
  @@index([accountCode, createdAt])
}
```

3. API Specifications

Method	Route	Purpose	Role Gate (RBAC)
GET	/api/v1/ledger	Query posted ledger.	ledger.view
GET	/api/v1/ledger/transactions/:id	Retrieve balanced group.	ledger.view
GET	/api/v1/ledger/reconciliation	Run/read reconciliation.	ledger.reconcile
POST	/api/v1/ledger/compensating-adjustments	Create linked adjustment.	Owner and ledger.adjust

4. Verification & Testing Tasks

- ☐ Mandatory enterprise gate: pass service integration with Testcontainers, consumer/provider contract compatibility, Playwright API E2E for externally reachable workflows, tenant-isolation negative tests, idempotency/redelivery tests, and observability assertions where applicable.

Double-Entry Tests

- ☐ Every posted transaction balances.
- ☐ Reject zero/negative/unbalanced entries.

Immutability Tests

- ☐ Update/delete of POSTED data is blocked.

Reconciliation Tests

- ☐ Match each source type to ledger totals.
- ☐ Detect injected mismatch.

Isolation Checks

- ☐ Concurrent posting does not duplicate.
- ☐ All queries enforce organization scope.

Module Specification: Razorpay Payments & Refunds

Overview & Metadata

Module Path	apps/payment-service/src/modules/payments/
Default Assignee/Role	Unassigned - Payment Integrations
Initial Status	Pending
Core Responsibility	Organization-connected Razorpay checkout, signed webhooks, direct settlement, fees, reconciliation, and limited refunds.

1. Functional Requirements & Business Logic

- 154. Each organization connects its own Razorpay account for direct settlement; the platform does not redistribute funds in the MVP.
- 155. Donor chooses gross amount. Receipt shows gross amount; organization bears gateway fee and fee tax.
- 156. Create payment order server-side and bind it to a pending donation installment.
- 157. Never confirm from frontend success alone; require server-side signature or trusted signed webhook.
- 158. Webhook processing is idempotent by provider event ID and payment ID.
- 159. Lifecycle is CREATED -> PENDING -> AUTHORIZED -> CAPTURED -> VERIFIED -> DONATION_CONFIRMED, with FAILED, EXPIRED, and ABANDONED.
- 160. Failed/expired/abandoned payments create no receipt and need no refund.
- 161. Refunds are limited to duplicate or incorrect captured online payments and may be partial or full.
- 162. Cumulative successful refunds cannot exceed captured amount.
- 163. Refunds post compensating ledger entries and update receipt verification state.
- 164. Encrypt all organization Razorpay credentials and redact provider secrets from logs.

2. Data Model Schema (PostgreSQL / Prisma ORM)

// Service-owned Prisma schema. Cross-service references are scalar UUIDs; direct ORM joins and foreign keys across services are prohibited.

```
enum OnlinePaymentStatus {
  CREATED PENDING AUTHORIZED CAPTURED VERIFIED DONATION_CONFIRMED
  FAILED EXPIRED ABANDONED
}
enum RefundStatus { REQUESTED PROCESSING PARTIALLY_REFUNDED REFUNDED FAILED REJECTED }
```

```
model RazorpayConnection {
  id          String  @id @default(uuid()) @db.Uuid
  organizationId String @unique @db.Uuid // external tenant-service aggregate ID; no cross-service FK
  keyIdMasked String  @db.VarChar(80)
  encryptedKeyId String
  encryptedKeySecret String
  encryptedWebhookSecret String
  isActive    Boolean @default(true)
  connectedByUserId String @db.Uuid
  connectedAt DateTime @default(now())
  updatedAt   DateTime @updatedAt
}
```

```
model OnlinePayment {
  id          String  @id @default(uuid()) @db.Uuid
  organizationId String @db.Uuid // external tenant-service aggregate ID; no cross-service FK
  eventId     String  @db.Uuid
  donationInstallmentId String @unique @db.Uuid
  providerOrderId String @unique @db.VarChar(150)
  providerPaymentId String? @unique @db.VarChar(150)
  status      OnlinePaymentStatus @default(CREATED)
  grossAmountPaise BigInt
  gatewayFeePaise BigInt @default(0)
  gatewayTaxPaise BigInt @default(0)
  netSettlementPaise BigInt?
  currency    String @default("INR") @db.Char(3)
  signatureVerifiedAt DateTime?
```

```

capturedAt      DateTime?
verifiedAt      DateTime?
settlementReference String? @db.VarChar(150)
failureCode     String? @db.VarChar(100)
failureDescription String? @db.Text
createdAt       DateTime @default(now())
updatedAt       DateTime @updatedAt
refunds         OnlinePaymentRefund[]
@@index([organizationId, eventId, status])
@@index([capturedAt])
}

model PaymentWebhookEvent {
  id          String @id @default(uuid()) @db.Uuid
  organizationId String @db.Uuid // external tenant-service aggregate ID; no cross-service FK
  providerEventId String @unique @db.VarChar(150)
  eventType   String @db.VarChar(100)
  signatureValid Boolean
  payloadHash String @db.VarChar(128)
  processingStatus String @db.VarChar(40)
  processedAt DateTime?
  failureCode String?
  receivedAt DateTime @default(now())
  @@index([organizationId, receivedAt])
}

model OnlinePaymentRefund {
  id          String @id @default(uuid()) @db.Uuid
  onlinePaymentId String @db.Uuid
  providerRefundId String? @unique @db.VarChar(150)
  amountPaise BigInt
  status      RefundStatus @default(REQUESTED)
  reason      String @db.Text
  requestedByUserId String @db.Uuid
  approvedByUserId String @db.Uuid
  ledgerTransactionId String? @unique @db.Uuid
  requestedAt DateTime @default(now())
  completedAt DateTime?
  failureDescription String? @db.Text
  payment OnlinePayment @relation(fields: [onlinePaymentId], references: [id], onDelete: Restrict)
  @@index([onlinePaymentId, status])
}

```

3. API Specifications

Method	Route	Purpose	Role Gate (RBAC)
POST	/api/v1/payments/razorpay/orders	Create provider order.	Donor or donation.create
POST	/api/v1/payments/razorpay/verify	Verify browser callback signature.	Authenticated donor
POST	/api/v1/webhooks/razorpay:organizationCode	Process signed webhook.	Public signed webhook
GET	/api/v1/payments/:id	Retrieve safe payment status.	payment.view or owning donor
POST	/api/v1/payments/:id/refunds	Request eligible refund.	payment.refund
GET	/api/v1/payments/reconciliation	View payment/fee/settlement reconciliation.	payment.reconcile
PATCH	/api/v1/settings/razorpay	Connect or rotate credentials.	Owner
GET	/api/v1/settings/reference-data:catalog	Read active localized reference data.	Active member
POST	/api/v1/settings/reference-data	Create/version organization reference data.	Owner

Method	Route	Purpose	Role Gate (RBAC)
GET	/api/v1/platform/feature-flags	List platform feature flags.	Platform Super Admin
PATCH	/api/v1/platform/feature-flags/:key	Version and roll out a feature flag.	Platform Super Admin

4. Verification & Testing Tasks

- ☐ Mandatory enterprise gate: pass service integration with Testcontainers, consumer/provider contract compatibility, Playwright API E2E for externally reachable workflows, tenant-isolation negative tests, idempotency/redelivery tests, and observability assertions where applicable.

Signature & Idempotency

- ☐ Validate correct/incorrect signatures.
- ☐ Replay webhook and create only one receipt/ledger posting.

Financial Tests

- ☐ Receipt amount equals gross; net equals gross-fee-tax.
- ☐ Refund total never exceeds captured amount.

Integration Tests

- ☐ Captured webhook completes donation flow.
- ☐ Refund completion posts compensation and receipt state.

Security Checks

- ☐ Secrets absent from logs/responses.
- ☐ Tenant webhook cannot resolve another tenant payment.

Module Specification: Public Event Transparency

Overview & Metadata

Module Path	apps/public-query-service/src/modules/public-events/
Default Assignee/Role	Unassigned - Public Platform
Initial Status	Pending
Core Responsibility	Public event pages, donor visibility, live totals, redacted vouchers, and immutable financial snapshots.

1. Functional Requirements & Business Logic

- 165.Allow Owner or authorized public-page publisher to enable and publish an event transparency page.
- 166.Public data may include confirmed monetary total, verified in-kind value, paid expenses, approved outstanding expenses, available balance, projected balance, donor name and amount, category summaries, and redacted vouchers.
- 167.Default donor visibility is NAME_AND_AMOUNT. Donor may select PRIVATE; organization cannot override PRIVATE.
- 168.Never expose donor mobile/email, internal IDs, provider IDs, private notes, or original voucher links.
- 169.Original vouchers remain private; only approved redacted derivatives can be public.
- 170.Support live totals from confirmed sources and immutable versioned snapshots.
- 171.Published snapshots cannot be edited or deleted; later publication creates a new version.
- 172.Financial closure prepares a final snapshot but Admin approval is required to publish.
- 173.Use non-enumerable slugs, public rate limits, caching, and paginated donor lists.
- 174.Consent and publication changes are audited.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
// Service-owned Prisma schema. Cross-service references are scalar UUIDs; direct ORM joins and foreign keys across services are prohibited.
enum PublicVisibility { NAME_AND_AMOUNT PRIVATE }
enum PublicPageStatus { DISABLED DRAFT PUBLISHED }

model PublicEventPage {
  id          String      @id @default(uuid()) @db.Uuid
  organizationId String    @db.Uuid // external tenant-service aggregate ID; no cross-service FK
  eventId     String      @unique @db.Uuid // external event-service aggregate ID; no cross-service FK
  publicSlug  String      @unique @db.VarChar(120)
  status      PublicPageStatus @default(DRAFT)
  showLiveTotals Boolean    @default(true)
  showDonorList Boolean     @default(true)
  showExpenseSummary Boolean @default(true)
  showRedactedBills Boolean  @default(true)
  approvedByUserId String?   @db.Uuid
  publishedAt DateTime?
  createdAt   DateTime      @default(now())
  updatedAt   DateTime      @updatedAt
  snapshots   PublicFinancialSnapshot[]
  @@index([organizationId, status])
}

model PublicFinancialSnapshot {
  id          String      @id @default(uuid()) @db.Uuid
  publicEventPageId String @db.Uuid
  version     Int
  confirmedDonationPaise BigInt
  verifiedInKindValuePaise BigInt
  paidExpensePaise BigInt
  outstandingApprovedPaise BigInt
  availableBalancePaise BigInt
  projectedBalancePaise BigInt
  categorySummary Json
  donorPublicationSnapshot Json
}
```

```

redactedDocumentIds  Json
publishedByUserId    String @db.Uuid
publishedAt          DateTime @default(now())
page                 PublicEventPage @relation(fields: [publicEventPageId], references: [id], onDelete: Restrict)
@@unique([publicEventPageId, version])
}

model PublicDonorPublication {
  id          String      @id @default(uuid()) @db.Uuid
  donationId  String      @unique @db.Uuid
  publicEventPageId String @db.Uuid
  visibility   PublicVisibility
  donorSelected Boolean    @default(false)
  approvedByUserId String? @db.Uuid
  approvedAt   DateTime?
  publishedAt   DateTime?
  @@index([publicEventPageId, visibility])
}

```

3. API Specifications

Method	Route	Purpose	Role Gate (RBAC)
GET	/api/v1/public/events/:slug	View public event and live totals.	Public
GET	/api/v1/public/events/:slug/donors	View approved donor name/amount list.	Public
GET	/api/v1/public/events/:slug/snapshots	List published snapshots.	Public
GET	/api/v1/events/:eventId/public-settings	Read internal public settings.	public-page.view
PATCH	/api/v1/events/:eventId/public-settings	Configure page.	public-page.manage
POST	/api/v1/events/:eventId/public-publish	Approve and publish page.	Owner or public-page.publish
POST	/api/v1/events/:eventId/public-snapshots	Publish immutable snapshot.	Owner or public-page.publish
POST	/api/v1/documents/:id/publish-redacted	Publish approved derivative.	public-page.publish

4. Verification & Testing Tasks

- ☐ Mandatory enterprise gate: pass service integration with Testcontainers, consumer/provider contract compatibility, Playwright API E2E for externally reachable workflows, tenant-isolation negative tests, idempotency/redelivery tests, and observability assertions where applicable.

Privacy Tests

- ☐ PRIVATE donations never appear.
- ☐ No contact/provider/private document data is exposed.

Snapshot & Math

- ☐ Snapshot totals reconcile with ledger/source.
- ☐ Earlier versions remain unchanged.

Security & Performance

- ☐ Test public rate limiting, caching, slug safety, pagination.
- ☐ Publication requires explicit authorization.

Module Specification: Reports & Export Jobs

Overview & Metadata

Module Path	apps/reporting-service/src/modules/reports/
Default Assignee/Role	Unassigned - Business Intelligence
Initial Status	Pending
Core Responsibility	Thirteen MVP reports with event, organization, financial-year, date filters, and PDF/XLSX/CSV exports.

1. Functional Requirements & Business Logic

- 175. Provide Event Financial Summary, Donation Register, Donation by Type, Payment Mode Collection, Receipt Register, In-Kind Contribution, Expense Register, Expense Category, Outstanding Expense, Financial Account Balance, Account Ledger, Public Transparency, and Financial-Year Summary.
- 176. Support event and organization scope where applicable, financial year, date range, pagination, and stable sorting.
- 177. Only authorized organization users may view/export. Platform Super Admin cannot access tenant finance.
- 178. Exclude donor mobile/email from all standard reports and exports.
- 179. Calculate from frozen receipt/expense snapshots, confirmed payments, and posted ledger entries.
- 180. Small reports may be synchronous; large PDF/XLSX/CSV exports use BullMQ and private object storage.
- 181. Completed export files are immutable, expire by retention policy, and use signed URLs.
- 182. Default financial year is April 1 to March 31.
- 183. All aggregation uses integer pause and deterministic decimal handling.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
// Service-owned Prisma schema. Cross-service references are scalar UUIDs; direct ORM joins and foreign keys across services are prohibited.
enum ReportType {
  EVENT_FINANCIAL_SUMMARY DONATION_REGISTER DONATION_BY_TYPE
  PAYMENT_MODE_COLLECTION RECEIPT_REGISTER IN_KIND_CONTRIBUTION
  EXPENSE_REGISTER EXPENSE_CATEGORY OUTSTANDING_EXPENSE
  FINANCIAL_ACCOUNT_BALANCE ACCOUNT_LEDGER PUBLIC_TRANSPARENCY
  FINANCIAL_YEAR_SUMMARY
}
enum ExportFormat { PDF XLSX CSV }
enum ReportJobStatus { QUEUED PROCESSING COMPLETED FAILED EXPIRED }

model ReportExportJob {
  id          String      @id @default(uuid()) @db.Uuid
  organizationId String    @db.Uuid // external tenant-service aggregate ID; no cross-service FK
  eventId      String?     @db.Uuid
  requestedByUserId String  @db.Uuid
  reportType   ReportType
  format       ExportFormat
  status       ReportJobStatus @default(QUEUED)
  filters      Json
  snapshotCutoffAt DateTime
  outputDocumentId String?  @unique @db.Uuid
  rowCount     Int?
  failureCode   String?
  failureDescription String? @db.Text
  expiresAt    DateTime?
  createdAt    DateTime     @default(now())
  startedAt    DateTime?
  completedAt  DateTime?
  @@index([organizationId, reportType, createdAt])
  @@index([status, createdAt])
}
```

3. API Specifications

Method	Route	Purpose	Role Gate (RBAC)
GET	/api/v1/reports/event-financial-summary	Event financial summary.	report.view
GET	/api/v1/reports/donations	Donation/type/mode reports.	report.view
GET	/api/v1/reports/receipts	Receipt register.	report.view
GET	/api/v1/reports/in-kind	In-kind report.	report.view
GET	/api/v1/reports/expenses	Expense/category/outstanding reports.	report.view
GET	/api/v1/reports/accounts	Account balance/ledger reports.	report.view
GET	/api/v1/reports/public-transparency	Transparency report.	report.view
GET	/api/v1/reports/financial-year	Financial-year summary.	report.view
POST	/api/v1/reports/exports	Queue PDF/XLSX/CSV.	report.export
GET	/api/v1/reports/exports/:id	Read status/download URL.	report.export and requester/Owner

4. Verification & Testing Tasks

- ☐ Mandatory enterprise gate: pass service integration with Testcontainers, consumer/provider contract compatibility, Playwright API E2E for externally reachable workflows, tenant-isolation negative tests, idempotency/redelivery tests, and observability assertions where applicable.

Calculation Tests

- ☐ Donation/expense/account totals reconcile with sources and ledger.
- ☐ Validate available/projected balance formulas.

Exporter Tests

- ☐ Validate CSV escaping, XLSX structure, and PDF pagination.
- ☐ Large exports run in worker.

Historical Tests

- ☐ Master-data changes cannot alter historical reports.

RBAC & Privacy

- ☐ Unauthorized and Platform Super Admin get 403.
- ☐ No donor contact fields in exports.

Module Specification: Multilingual Notifications

Overview & Metadata

Module Path	apps/notification-service/src/modules/notifications/
Default Assignee/Role	Unassigned - Integrations
Initial Status	Pending
Core Responsibility	In-app, email, WhatsApp, and OTP SMS delivery in English, Marathi, and Hindi without automatic retry.

1. Functional Requirements & Business Logic

- 184.Support IN_APP, EMAIL, WHATSAPP, and SMS; SMS is used only for OTP.
- 185.Render in user preferred English, Marathi, or Hindi with organization default and English fallback.
- 186.WhatsApp must use approved templates and store template version.
- 187.Notify for OTP, member invite, donation submitted/confirmed, cheque cleared/bounced, receipt generated/cancelled/replaced, online payment failed, refund completed, expense submitted/approved/rejected/awaiting approval, event financial closure, and public report publication.
- 188.Lifecycle is QUEUED -> SENT -> DELIVERED or FAILED.
- 189.Failed deliveries are stored and never retried automatically.
- 190.Authorized users may manually resend eligible failed non-OTP notifications as a new linked attempt.
- 191.Never include secrets, full bank data, private voucher URLs, or hidden donor data.
- 192.Use BullMQ for email/WhatsApp; record provider status for every channel.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
// Service-owned Prisma schema. Cross-service references are scalar UUIDs; direct ORM joins and foreign keys across services are prohibited.
enum NotificationChannel { IN_APP EMAIL WHATSAPP SMS }
enum NotificationStatus { QUEUED SENT DELIVERED FAILED }
enum NotificationEventType {
  OTP_GENERATED MEMBER_INVITED DONATION_SUBMITTED DONATION_CONFIRMED
  CHEQUE_CLEARED CHEQUE_BOUNCED RECEIPT_GENERATED
  RECEIPT_CANCELLED_OR_REPLACED ONLINE_PAYMENT_FAILED REFUND_COMPLETED
  EXPENSE_SUBMITTED EXPENSE_APPROVED EXPENSE_REJECTED
  EXPENSE_AWAITING_ADDITIONAL_APPROVAL EVENT_FINANCIALLY_CLOSED
  PUBLIC_REPORT_PUBLISHED
}

model Notification {
  id          String          @id @default(uuid()) @db.Uuid
  organizationId String?      @db.Uuid
  recipientUserId String      @db.Uuid // external identity-service aggregate ID; no cross-service FK
  eventType   NotificationEventType
  channel     NotificationChannel
  status      NotificationStatus @default(QUEUED)
  locale      PreferredLanguage
  templateCode String          @db.VarChar(100)
  templateVersion Int
  destinationMasked String?    @db.VarChar(150)
  payloadSnapshot Json
  providerMessageId String?    @db.VarChar(150)
  providerStatus String?      @db.VarChar(80)
  failureCode  String?        @db.VarChar(100)
  failureDescriptionSafe String? @db.Text
  originalNotificationId String? @db.Uuid
  queuedAt    DateTime        @default(now())
  sentAt      DateTime?
  deliveredAt DateTime?
  failedAt    DateTime?
  @@index([recipientUserId, status, queuedAt])
  @@index([organizationId, eventType, queuedAt])
}
```

```
model NotificationTemplate {
  id          String      @id @default(uuid()) @db.Uuid
  code        String      @db.VarChar(100)
  channel      NotificationChannel
  locale       PreferredLanguage
  version      Int
  subject      String?     @db.VarChar(250)
  bodyTemplate String      @db.Text
  providerTemplateName String? @db.VarChar(150)
  isApproved   Boolean     @default(false)
  isActive     Boolean     @default(false)
  createdAt    DateTime    @default(now())
  @@unique([code, channel, locale, version])
  @@index([code, isActive])
}
```

3. API Specifications

Method	Route	Purpose	Role Gate (RBAC)
GET	/api/v1/notifications	List current user's in-app notifications.	Authenticated
PATCH	/api/v1/notifications/:id/read	Mark notification read.	Recipient
GET	/api/v1/notifications/deliveries	View organization delivery history.	notification.view
POST	/api/v1/notifications/:id/resend	Manually resend eligible failure.	notification.resend
GET	/api/v1/settings/notification-templates	List active template mappings.	Owner
PATCH	/api/v1/settings/notification-preferences	Update language/channel preference.	Authenticated

4. Verification & Testing Tasks

- ☐ Mandatory enterprise gate: pass service integration with Testcontainers, consumer/provider contract compatibility, Playwright API E2E for externally reachable workflows, tenant-isolation negative tests, idempotency/redelivery tests, and observability assertions where applicable.

Localization Tests

- ☐ Render every event template in all three languages.
- ☐ Verify fallback and placeholder completeness.

Delivery Tests

- ☐ Failures do not auto-retry.
- ☐ Manual resend creates linked attempt.

Security & Privacy

- ☐ Payload scan excludes secrets/private data.
- ☐ Users access only own in-app notifications.

Module Specification: Documents, Vouchers & Redaction

Overview & Metadata

Module Path	apps/document-service/src/modules/documents/
Default Assignee/Role	Unassigned - Infrastructure
Initial Status	Pending
Core Responsibility	Private storage, presigned access, file validation, malware scanning, retention, and public redacted derivatives.

1. Functional Requirements & Business Logic

- 193.Allow PDF, JPG, JPEG, PNG, and WebP and validate actual MIME signature and extension.
- 194.Maximum general file size is 10 MB; logos 5 MB; expenses and in-kind contributions allow up to five attachments.
- 195.Use private tenant-scoped object keys and no permanent public URL for originals.
- 196.Presigned upload/download URLs are short-lived and bound to expected type, size, checksum, user, organization, and purpose.
- 197.Run malware scanning before AVAILABLE. Quarantined/rejected files cannot be attached.
- 198.Original vouchers remain private; publication uses a generated, approved redacted derivative.
- 199.Redact bank/PAN/contact/address/internal notes/sensitive signatures while preserving original bytes.
- 200.Only unreferenced draft documents may be deleted after retention; finalized financial documents are immutable.
- 201.Log sensitive document access.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
// Service-owned Prisma schema. Cross-service references are scalar UUIDs; direct ORM joins and foreign keys across services are prohibited.
enum DocumentPurpose {
  ORGANIZATION_LOGO REGISTRATION_CERTIFICATE RECEIPT_PDF
  EXPENSE_VOUCHER IN_KIND_ATTACHMENT REPORT_EXPORT
  PUBLIC_REDACTED_VOUCHER AUTHORIZED_SIGNATURE
}
enum DocumentStatus {
  UPLOAD_PENDING SCANNING AVAILABLE QUARANTINED REJECTED DELETED
}

model DocumentAsset {
  id          String      @id @default(uuid()) @db.Uuid
  organizationId String?   @db.Uuid // external tenant-service aggregate ID; no cross-service FK
  ownerUserId  String      @db.Uuid
  purpose      DocumentPurpose
  status       DocumentStatus @default(UPLOAD_PENDING)
  objectKey    String      @unique @db.VarChar(500)
  originalFileName String  @db.VarChar(255)
  mimeType     String      @db.VarChar(100)
  fileSizeBytes BigInt
  sha256       String      @db.Char(64)
  malwareScanResult String? @db.VarChar(80)
  isImmutable  Boolean      @default(false)
  immutableAt  DateTime?
  retentionUntil DateTime?
  uploadedAt   DateTime?
  createdAt    DateTime     @default(now())
  @@index([organizationId, purpose, status])
  @@index([ownerUserId, createdAt])
}

model RedactedDocument {
  id          String      @id @default(uuid()) @db.Uuid
  organizationId String   @db.Uuid // external tenant-service aggregate ID; no cross-service FK
  sourceDocumentId String @db.Uuid
  redactedDocumentId String @unique @db.Uuid
  redactionProfile String @db.VarChar(100)
  redactionSummary Json
```

```
generatedByUserId String @db.Uuid
approvedByUserId String? @db.Uuid
generatedAt      DateTime @default(now())
approvedAt      DateTime?
publishedAt      DateTime?
@@unique([sourceDocumentId, redactedDocumentId])
@@index([organizationId, approvedAt])
}
```

3. API Specifications

Method	Route	Purpose	Role Gate (RBAC)
POST	/api/v1/documents/presign-upload	Create upload instructions.	Authenticated with purpose permission
POST	/api/v1/documents/:id/complete	Verify upload and scan.	Document owner
GET	/api/v1/documents/:id/download	Create signed download URL.	document.view and resource access
DELETE	/api/v1/documents/:id	Delete unreferenced draft.	Owner or document.manage
POST	/api/v1/documents/:id/redact	Generate redacted derivative.	document.redact
POST	/api/v1/documents/:id/redaction-approve	Approve public derivative.	Owner or public-page.publish

4. Verification & Testing Tasks

- ☐ Mandatory enterprise gate: pass service integration with Testcontainers, consumer/provider contract compatibility, Playwright API E2E for externally reachable workflows, tenant-isolation negative tests, idempotency/redelivery tests, and observability assertions where applicable.

Validation Tests

- ☐ Reject spoofed type, oversize, unsupported file, checksum mismatch.
- ☐ Enforce attachment limits.

Security Tests

- ☐ Signed URLs expire and are tenant-scoped.
- ☐ Quarantined files cannot be used.

Immutability & Redaction

- ☐ Finalized document cannot be deleted.
- ☐ Redaction leaves source unchanged.

Module Specification: Append-Only Audit Logs

Overview & Metadata

Module Path	apps/audit-service/src/modules/audit/
Default Assignee/Role	Unassigned - Security & Audit
Initial Status	Pending
Core Responsibility	Immutable evidence for security, financial, approval, publication, and configuration actions.

1. Functional Requirements & Business Logic

- 202.Audit organization status changes, donor merge, memberships, ownership transfer, roles, events, donation confirmation/cancellation, cheque events, receipt changes, in-kind verification, expenses, refunds, transfers, opening balances, publication, and settings.
- 203.Capture scope, organization/event, actor, approver, reason, target, before/after snapshots, request ID, IP hash, user agent, and timestamp.
- 204.Expose create and read only; no update or delete endpoints.
- 205.Protect against ORM mutation with restricted repositories/database permissions and optional hash chaining.
- 206.Owner and Auditor with audit.view may read organization logs. Platform Super Admin reads only platform-scope logs.
- 207.Platform audit projections must not expose tenant-private financial snapshots.
- 208.Retain for the organization lifetime and archive rather than delete.
- 209.Redact passwords, OTPs, tokens, and unmasked provider credentials from snapshots.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
enum AuditScope { PLATFORM ORGANIZATION }

model AuditLog {
  id          String      @id @default(uuid()) @db.Uuid
  scope       AuditScope
  organizationId String?   @db.Uuid
  eventId     String?     @db.Uuid
  actionType  String      @db.VarChar(120)
  performedByUserId String @db.Uuid
  approvedByUserId String? @db.Uuid
  reason      String?     @db.Text
  targetType  String      @db.VarChar(80)
  targetId    String      @db.Uuid
  beforeSnapshot Json?
  afterSnapshot Json?
  requestId   String      @db.VarChar(100)
  ipAddressHash String?   @db.VarChar(128)
  userAgent   String?     @db.VarChar(500)
  previousHash String?    @db.Char(64)
  recordHash  String      @unique @db.Char(64)
  createdAt   DateTime    @default(now())
  @@index([scope, organizationId, createdAt])
  @@index([eventId, createdAt])
  @@index([targetType, targetId])
  @@index([performedByUserId, createdAt])
}
```

3. API Specifications

Method	Route	Purpose	Role Gate (RBAC)
GET	/api/v1/audit-logs	Query organization audit trail.	Owner or audit.view
GET	/api/v1/audit-logs/:id	Retrieve one organization audit record.	Owner or audit.view
GET	/api/v1/platform/audit-logs	Query platform-scope audit.	Platform Super Admin

4. Verification & Testing Tasks

- ☐ Mandatory enterprise gate: pass service integration with Testcontainers, consumer/provider contract compatibility, Playwright API E2E for externally reachable workflows, tenant-isolation negative tests, idempotency/redelivery tests, and observability assertions where applicable.

Immutability Tests

- ☐ PUT/PATCH/DELETE unavailable.
- ☐ Repository mutation is blocked.

Integrity Tests

- ☐ Validate record hash/chain.
- ☐ Secrets are removed from snapshots.

RBAC Tests

- ☐ Ordinary members cannot read logs.
- ☐ Platform views redact tenant-private finance.

Module Specification: Organization & Platform Settings

Overview & Metadata

Module Path	apps/tenant-service/src/modules/settings/
Default Assignee/Role	Unassigned - Core Configuration
Initial Status	Pending
Core Responsibility	Versioned preferences for language, financial year, invitations, in-kind valuation, receipt templates, visibility, and platform limits.

1. Functional Requirements & Business Logic

- 210.Maintain one current organization settings record plus immutable history.
- 211.Configure default/enabled languages, Asia/Kolkata timezone, financial-year start, invitation acceptance, public visibility default, in-kind valuation, receipt template, Collector-bound books, and notification channels.
- 212.Default donor visibility is NAME_AND_AMOUNT; donor PRIVATE choice cannot be overridden.
- 213.In-kind inclusion affects contribution statistics only and never financial balance.
- 214.Only Owner mutates organization settings; active members read safe workflow settings.
- 215.Platform settings manage event types, file limits, templates, and future feature flags.
- 216.Changes apply prospectively and never alter receipts, submitted expenses, policy snapshots, or public snapshots.
- 217.Sensitive changes require reason and audit snapshot.
- 218.Configuration updates use optimistic locking, effective dates, validation schemas, audit snapshots, and a config.changed.v1 event for cache invalidation.
- 219.Stable lifecycle invariants, security protocol values, and contract version enums remain code-controlled. Changing them requires an ADR, compatibility analysis, and coordinated contract release.
- 220.All tenant-specific thresholds, event types, categories, units, upload limits, receipt and notification templates, report labels, provider options, and feature availability are resolved through versioned configuration/reference data; they must not be hardcoded in controllers or domain services.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
model OrganizationSettings {
  id          String      @id @default(uuid()) @db.Uuid
  organizationId String    @unique @db.Uuid
  version     Int         @default(1)
  defaultLanguage PreferredLanguage @default(EN)
  enabledLanguages PreferredLanguage[]
  timezone    String      @default("Asia/Kolkata") @db.VarChar(60)
  financialYearStartMonth Int @default(4)
  financialYearStartDay Int @default(1)
  invitationAcceptanceRequired Boolean @default(false)
  defaultDonorVisibility PublicVisibility @default(NAME_AND_AMOUNT)
  inKindValueRequired Boolean @default(false)
  includeInKindInContributionTotal Boolean @default(true)
  collectorBoundReceiptBooks Boolean @default(false)
  receiptTemplateCode String @default("STANDARD_TRILINGUAL") @db.VarChar(100)
  authorizedSignatureDocumentId String? @db.Uuid
  whatsappEnabled Boolean @default(true)
  emailEnabled Boolean @default(true)
  inAppEnabled Boolean @default(true)
  updatedByUserId String @db.Uuid
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  organization Organization @relation(fields: [organizationId], references: [id], onDelete: Restrict)
}

model OrganizationSettingsHistory {
  id String @id @default(uuid()) @db.Uuid
  organizationId String @db.Uuid
  version Int
  snapshot Json
```

```

    changedByUserId String @db.Uuid
    changeReason    String @db.Text
    createdAt       DateTime @default(now())
    @@unique([organizationId, version])
}

model PlatformSettings {
    id          String @id @default(uuid()) @db.Uuid
    version     Int    @unique
    supportedEventTypes Json
    maxFileSizeBytes BigInt @default(10485760)
    maxLogoSizeBytes BigInt @default(5242880)
    maxAttachmentsPerEntity Int @default(5)
    featureFlags Json
    updatedByUserId String @db.Uuid
    createdAt     DateTime @default(now())
}

enum ConfigurationScope { PLATFORM ORGANIZATION EVENT }
enum ConfigurationValueType { STRING NUMBER BOOLEAN JSON }
model ConfigurationEntry {
    id          String @id @default(uuid()) @db.Uuid
    scope       ConfigurationScope
    scopeId     String? @db.Uuid
    key         String @db.VarChar(160)
    valueType   ConfigurationValueType
    valueJson   Json
    schemaVersion Int @default(1)
    version     Int @default(1)
    isSensitive Boolean @default(false)
    effectiveFrom DateTime @default(now())
    effectiveTo DateTime?
    changedByUserId String @db.Uuid
    changeReason String @db.Text
    createdAt DateTime @default(now())
    @@unique([scope, scopeId, key, version])
    @@index([scope, scopeId, key, effectiveFrom])
}

model FeatureFlag {
    id          String @id @default(uuid()) @db.Uuid
    key         String @unique @db.VarChar(160)
    description String @db.Text
    enabled     Boolean @default(false)
    scopeRules  Json?
    rolloutPercent Int @default(0)
    ownerTeam   String @db.VarChar(100)
    expiresAt   DateTime?
    changedByUserId String @db.Uuid
    changeReason String @db.Text
    updatedAt   DateTime @updatedAt
}

model ReferenceDataValue {
    id          String @id @default(uuid()) @db.Uuid
    organizationId String? @db.Uuid // null means platform default
    catalog     String @db.VarChar(100)
    code        String @db.VarChar(80)
    labels      Json
    metadata    Json?
    isActive    Boolean @default(true)
    effectiveFrom DateTime @default(now())
    effectiveTo DateTime?
    createdAt   DateTime @default(now())
    @@unique([organizationId, catalog, code, effectiveFrom])
    @@index([catalog, isActive])
}

```

3. API Specifications

Method	Route	Purpose	Role Gate (RBAC)
GET	/api/v1/settings	Retrieve safe organization settings.	Active member
PATCH	/api/v1/settings	Update organization settings.	Owner
GET	/api/v1/settings/history	View settings history.	Owner or audit.view
GET	/api/v1/platform/settings	Retrieve platform settings.	Platform Super Admin
PATCH	/api/v1/platform/settings	Create new platform settings version.	Platform Super Admin

4. Verification & Testing Tasks

- ☐ Mandatory enterprise gate: pass service integration with Testcontainers, consumer/provider contract compatibility, Playwright API E2E for externally reachable workflows, tenant-isolation negative tests, idempotency/redelivery tests, and observability assertions where applicable.

Validation Tests

- ☐ Validate timezone, financial-year date, language set, and limits.

Historical Isolation

- ☐ Settings change does not alter frozen transactions/documents.

RBAC & Audit

- ☐ Only Owner mutates organization settings.
- ☐ Sensitive changes create safe before/after audit.

System-Wide Acceptance Criteria

- ☐ Every domain service builds, tests, migrates, deploys, scales, observes, and rolls back independently; no runtime dependency on another service source package exists.
- ☐ No service reads or writes another service database. Cross-service data is accessed only through versioned contracts or owned read models.
- ☐ Transactional outbox/inbox and idempotency prevent duplicate ledger, receipt, refund, notification, and audit side effects under event redelivery.
- ☐ Nx boundary rules, CODEOWNERS, generated contracts, and affected CI prove package reusability without architecture erosion.
- ☐ Playwright API E2E and critical browser journeys pass with retained traces; contract, performance, security, and resilience gates are green.
- ☐ All production images are non-root, scanned, signed, SBOM-attested, and deployed by Helm with probes, policies, autoscaling, and tested rollback.
- ☐ No tenant-specific business behavior or secret is hardcoded; versioned configuration, reference data, templates, and feature flags are validated and audited.
- ☐ OpenTelemetry traces connect Gateway, gRPC calls, NATS events, BullMQ jobs, Prisma operations, and provider calls using one correlation context.
- ☐ No cross-organization data access is possible through identifiers, query filters, exports, files, or public/private endpoint confusion.
- ☐ Every confirmed monetary movement is posted to the immutable ledger exactly once.
- ☐ Every receipt has a unique event-year number, frozen snapshot, verifiable QR token, and immutable PDF lifecycle.
- ☐ Every approved expense follows a frozen policy, distinct approvals, mandatory voucher, and exact payment balance.
- ☐ No issued receipt, approved expense, posted ledger entry, public snapshot, or audit record can be edited or deleted.
- ☐ Razorpay webhooks are verified and idempotent; frontend callbacks alone never confirm donations.
- ☐ Public pages respect donor PRIVATE choices and expose only approved redacted voucher derivatives.
- ☐ All thirteen MVP reports reconcile to frozen source and ledger data and exclude donor contact information.
- ☐ English, Marathi, and Hindi receipts and notifications pass glyph and layout verification.
- ☐ Backups, restores, migrations, logs, metrics, alerts, and runbooks are verified before production release.

Enterprise Standards and Official Reference Baseline

- NestJS Microservices: <https://docs.nestjs.com/microservices/basics>
- Nx Monorepo Concepts and Boundary Governance: <https://nx.dev/docs/concepts/decisions/what-is-a-monorepo>
- Playwright API Testing: <https://playwright.dev/docs/api-testing>
- Kubernetes Production and Deployment Documentation: <https://kubernetes.io/docs/setup/> and <https://kubernetes.io/docs/reference/kubernetes-api/apps/deployment-v1/>
- NATS JetStream Reliability and Consumers: <https://docs.nats.io/nats-concepts/jetstream>
- OpenTelemetry Documentation: <https://opentelemetry.io/docs/>

Enterprise Revision 2.1 - Domain Expansion

This section is part of the normative Pauti Pustak backend architecture. It adds the missing volunteer, personalized billing, payment collection, unified contribution, and receipt-generation capabilities while retaining all non-conflicting Version 2.0 controls.

Normative Change Control

The requirements, schemas, APIs, events, permissions, workflows, and acceptance criteria in this revision are mandatory. Where a conflict exists, this section takes precedence over Version 2.0. Database migrations must preserve existing donation and receipt identifiers and expose compatibility adapters until all consumers migrate.

Updated Deployable Service Topology

Deployable Service	Owned Modules	Owned Persistence	Primary Interfaces	Scaling Profile
volunteer-service	volunteers, assignments, areas, activity history	volunteer_db	REST via gateway, gRPC, volunteer lifecycle events	Read-heavy; burst assignment/import operations
donor-service	donors, contributor-accounts, duplicate merge	donor_db	gRPC, donor and contributor events	Search/read-heavy
contribution-service	contributions, contribution-bills, collection-records, receipt-books, receipts, in-kind contributions	contribution_db	REST via gateway, gRPC, contribution/bill/collection/receipt events	Festival write peaks; PDF and batch workers
finance-service	vendors, approval-policies, expenses, accounts, ledger	finance_db	gRPC, ledger/expense/account events	Strict consistency
payment-service	Razorpay connections, payments, webhooks, refunds	payment_db	Webhook ingress, gRPC, payment events	Burst webhook load
reporting-service	reports, exports, volunteer/bill/collection read models	reporting_db	REST, NATS consumers, export workers	CPU/memory worker scaling

Updated Monorepo Paths

```
apps/  
  volunteer-service/  
  donor-service/  
    src/modules/contributor-accounts/  
  contribution-service/  
    src/modules/contributions/  
    src/modules/contribution-bills/  
    src/modules/collection-records/  
    src/modules/receipt-books/  
    src/modules/receipts/  
  reporting-service/  
libs/contracts/  
  volunteer/ contributor/ bill/ collection/ contribution/ receipt/
```

Cross-Service Ownership Rules

- Volunteer-service owns volunteer identity and assignment history. Contribution-service stores only frozen collector snapshots and opaque volunteer IDs.
- Donor-service owns canonical donor identity and contributor-account contact/address data. Issued bills and receipts retain immutable snapshots.
- Contribution-service owns bill numbers, collection verification, contribution confirmation, and receipt allocation. Finance-service owns ledger truth.
- No cross-service SQL, ORM relation, shared table, or foreign key is allowed. Existence checks use gRPC or authoritative lifecycle events.

Module Specification: Volunteer Management

Overview & Metadata

Module Path	apps/volunteer-service/src/modules/volunteers/
Default Assignee/Role	Unassigned - Volunteer Operations
Initial Status	Pending
Core Responsibility	Organization and event volunteer records, optional login linkage, privacy, lifecycle, documents, assignments, collection responsibility, and immutable participation history.

1. Functional Requirements & Business Logic

- 217. Maintain volunteers independently from system users; a volunteer may exist without login credentials.
- 218. Allow optional one-to-one linkage to a verified identity-service user. Linking never changes historical volunteer snapshots.
- 219. Support GENERAL, DONATION_COLLECTOR, EVENT_COORDINATOR, FINANCE_VOLUNTEER, DECORATION, FOOD_DISTRIBUTION, CROWD_MANAGEMENT, and CUSTOM volunteer types.
- 220. Capture full name, mobile, email, address, emergency contact, joining date, preferred language, profile document, identity-document references, and notes.
- 221. Volunteer lifecycle is DRAFT -> ACTIVE -> SUSPENDED -> INACTIVE. Reactivation requires volunteer.manage and an audit reason.
- 222. A volunteer may participate in multiple events and hold a different role in each event.
- 223. Assignments may target an event, area, route, society, village, ward, contributor portfolio, receipt book, or collection campaign.
- 224. Collection authority is granted only by an active event assignment plus required permissions; volunteer type alone is not authorization.
- 225. Deactivation blocks new assignments and collections but never removes historical assignments, bill ownership, collection records, or receipts.
- 226. Identity documents remain private and are accessed only through signed document-service URLs with purpose-based audit.
- 227. Volunteer search and exports must mask contact and identity data unless the caller has volunteer.sensitive.view.
- 228. Collector performance is computed from confirmed collections and reconciled settlements; manual performance totals are prohibited.
- 229. Bulk import requires validation preview, duplicate detection, row-level error output, and idempotent commit.
- 230. Every sensitive create/update/status/assignment action publishes an immutable audit event.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
enum VolunteerStatus { DRAFT ACTIVE SUSPENDED INACTIVE }
enum VolunteerType { GENERAL DONATION_COLLECTOR EVENT_COORDINATOR FINANCE_VOLUNTEER DECORATION FOOD_DISTRIBUTION CROWD_MANAGEMENT CUSTOM }

model Volunteer {
  id String @id @default(uuid()) @db.Uuid
  organizationId String @db.Uuid
  linkedUserId String? @unique @db.Uuid
  volunteerCode String @db.VarChar(30)
  type VolunteerType
  customTypeLabel String? @db.VarChar(100)
  status VolunteerStatus @default(DRAFT)
  fullName String @db.VarChar(160)
  mobileEncrypted String
  mobileHash String @db.VarChar(128)
  emailEncrypted String?
  preferredLanguage String @default("mr") @db.VarChar(10)
  addressSnapshot Json?
  emergencyContactEncrypted String?
  joinedOn DateTime?
  profileDocumentId String? @db.Uuid
  createdByUserId String @db.Uuid
  version Int @default(1)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  @@unique([organizationId, volunteerCode])
  @@index([organizationId, status, type])
  @@index([organizationId, mobileHash])
}
```

```
enum VolunteerAssignmentStatus { PLANNED ACTIVE COMPLETED CANCELLED }
enum AssignmentScopeType { EVENT AREA ROUTE SOCIETY VILLAGE WARD CONTRIBUTOR_PORTFOLIO RECEIPT_BOOK CAMPAIGN }

model VolunteerAssignment {
  id String @id @default(uuid()) @db.Uuid
  organizationId String @db.Uuid
  eventId String @db.Uuid
  volunteerId String @db.Uuid
  roleCode String @db.VarChar(80)
  scopeType AssignmentScopeType
  scopeReferenceId String? @db.VarChar(120)
  scopeSnapshot Json
  startsAt DateTime
  endsAt DateTime?
  status VolunteerAssignmentStatus @default(PLANNED)
  assignedByUserId String @db.Uuid
  endedByUserId String? @db.Uuid
  endReason String? @db.Text
  createdAt DateTime @default(now())
  @@index([organizationId, eventId, volunteerId, status])
  @@index([organizationId, eventId, scopeType, scopeReferenceId])
}
```

3. API Specifications

Method	Route	Purpose	Role Gate (RBAC)
GET	/api/v1/volunteers	List/search volunteers with masked fields.	volunteer.view
POST	/api/v1/volunteers	Create volunteer.	volunteer.create
GET	/api/v1/volunteers/:id	Read volunteer and assignment summary.	volunteer.view
PATCH	/api/v1/volunteers/:id	Update mutable profile fields with optimistic version.	volunteer.update
POST	/api/v1/volunteers/:id/activate	Activate volunteer.	volunteer.manage
POST	/api/v1/volunteers/:id/suspend	Suspend with reason.	volunteer.manage
POST	/api/v1/volunteers/:id/link-user	Link verified user identity.	volunteer.manage
POST	/api/v1/events/:eventId/volunteer-assignments	Create event/scope assignment.	volunteer.assignment.manage
PATCH	/api/v1/volunteer-assignments/:id	Update planned/active assignment.	volunteer.assignment.manage
POST	/api/v1/volunteer-assignments/:id/end	Complete/cancel assignment with reason.	volunteer.assignment.manage
POST	/api/v1/volunteers/imports	Validate and queue bulk import.	volunteer.import

4. Domain Events & Integrations

Event Subject	Producer	Primary Consumers
pauti.volunteer.volunteer.created.v1	volunteer-service	audit, reporting
pauti.volunteer.volunteer.status-changed.v1	volunteer-service	contribution, reporting, audit
pauti.volunteer.assignment.created.v1	volunteer-service	contribution, reporting, notifications
pauti.volunteer.assignment.ended.v1	volunteer-service	contribution, reporting, audit

5. Verification & Testing Tasks

- Tenant-isolation tests cover list, ID substitution, import files, signed documents, cache keys, and event consumers.
- Duplicate import replay does not create duplicate volunteers or assignments.
- Inactive/suspended volunteers cannot create collection records; historical records remain queryable.
- Overlapping assignment rules are deterministic and configurable by scope type.
- Sensitive fields are encrypted, masked in logs, and absent from unauthorized exports.
- Audit evidence captures actor, reason, before/after snapshot, correlation ID, and request source.

Module Specification: Contributor Accounts

Overview & Metadata

Module Path	apps/donor-service/src/modules/contributor-accounts/
Default Assignee/Role	Unassigned - Business Domain
Initial Status	Pending
Core Responsibility	Event-specific contribution accounts for individuals, families, households, shops, societies, sponsors, institutions, and government bodies, including billing and collector preferences.

1. Functional Requirements & Business Logic

- 233. Every contributor account references a canonical donor profile or organization identity; anonymous bill accounts are prohibited.
- 234. Support INDIVIDUAL, FAMILY_HOUSEHOLD, SHOP_BUSINESS, HOUSING_SOCIETY, INSTITUTION, SPONSOR, GOVERNMENT_BODY, and OTHER account types.
- 235. A canonical donor may have multiple event-specific contributor accounts when addresses, billing categories, or responsible contacts differ.
- 236. Capture event, display name, contact person, mobile/email, billing address, area/route/society/ward, language, contribution category, and assigned collector preference.
- 237. Contact and address changes affect future bills only. Issued bills and receipts retain frozen snapshots.
- 238. Allow category-based suggested amounts, fixed requested amounts, open-amount billing, or no-bill direct contribution.
- 239. Maintain previous-event contribution summaries as derived read models; do not copy mutable totals into new financial aggregates.
- 240. Duplicate detection uses normalized mobile, address, canonical donor, and organization-defined matching rules.
- 241. Merge operations require contributor.merge, a dry-run impact report, separate approval for accounts with issued bills, and immutable alias mapping.
- 242. Contributor privacy preference is authoritative for public donor display and cannot be overridden by organization defaults.
- 243. Assignments to collectors are prospective; reassignment never rewrites previously issued bills or confirmed collections.
- 244. Bulk import supports CSV/XLSX validation, preview, row errors, deduplication choices, and idempotent commit.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
enum ContributorAccountType { INDIVIDUAL FAMILY_HOUSEHOLD SHOP_BUSINESS HOUSING_SOCIETY INSTITUTION SPONSOR GOVERNMENT_BODY OTHER }
enum BillingMode { FIXED SUGGESTED OPEN_AMOUNT NO_BILL }
enum ContributorAccountStatus { ACTIVE INACTIVE MERGED }

model ContributorAccount {
  id String @id @default(uuid()) @db.Uuid
  organizationId String @db.Uuid
  eventId String @db.Uuid
  donorProfileId String @db.Uuid
  accountCode String @db.VarChar(40)
  type ContributorAccountType
  status ContributorAccountStatus @default(ACTIVE)
  displayName String @db.VarChar(200)
  contactName String? @db.VarChar(160)
  contactSnapshot Json
  billingAddressSnapshot Json
  areaCode String? @db.VarChar(80)
  routeCode String? @db.VarChar(80)
  categoryCode String? @db.VarChar(80)
  billingMode BillingMode @default(SUGGESTED)
  requestedAmountPaise BigInt?
  assignedVolunteerId String? @db.Uuid
  preferredLanguage String @default("mr") @db.VarChar(10)
  publicVisibility PublicVisibility @default(NAME_AND_AMOUNT)
  createdByUserId String @db.Uuid
  version Int @default(1)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  @@unique([organizationId, eventId, accountCode])
  @@index([organizationId, eventId, status, areaCode])
  @@index([organizationId, eventId, assignedVolunteerId])
}
```

3. API Specifications

Method	Route	Purpose	Role Gate
GET	/api/v1/events/:eventId/contributor-accounts	Search/filter accounts.	contributor.view
POST	/api/v1/events/:eventId/contributor-accounts	Create account.	contributor.create
PATCH	/api/v1/contributor-accounts/:id	Update future-facing details.	contributor.update
POST	/api/v1/contributor-accounts/:id/reassign	Assign/reassign collector prospectively.	contributor.assign
POST	/api/v1/contributor-accounts/merge-preview	Preview duplicate merge impact.	contributor.merge
POST	/api/v1/contributor-accounts/merge	Merge with approval/idempotency.	contributor.merge
POST	/api/v1/events/:eventId/contributor-imports	Validate/queue import.	contributor.import

4. Verification & Testing Tasks

- Issued bill snapshots do not change after contributor edits or merges.
- Cross-tenant and cross-event ID substitution returns 404/403 without existence leakage.
- Duplicate merge preserves aliases and all financial references.
- Collector reassignment affects only bills generated after effectiveAt unless an explicit unissued-bill reassignment is requested.

Module Specification: Personalized Contribution Bills

Overview & Metadata

Module Path	apps/contribution-service/src/modules/contribution-bills/
Default Assignee/Role	Unassigned - Finance Domain
Initial Status	Pending
Core Responsibility	Personalized bill drafting, atomic numbering, batch generation, immutable issue snapshots, PDF rendering, QR/payment links, delivery, status calculation, waiver, cancellation, and replacement.

1. Functional Requirements & Business Logic

248. Create a bill manually for one contributor account or generate bills in bulk by area, route, category, assignment, or validated import selection.
249. Bill lines support contribution, sponsorship, advertisement, membership, grant, and organization-defined categories.
250. A bill may contain one or more lines. Requested total is the sum of positive line amounts; open-amount bills may have a null requested total.
251. Public bill number format is configurable but must include organization code, event code, calendar/financial year, and an atomic sequence.
252. Number allocation occurs only during issue inside the same transaction that freezes contributor, event, organization, collector, template, amount, and payment-link snapshots.
253. Lifecycle is DRAFT -> GENERATED -> ISSUED -> PARTIALLY_PAID -> PAID, with OVERDUE, WAIVED, CANCELLED, RENDER_FAILED, and REPLACED alternatives.
254. DRAFT is editable. GENERATED permits preview. ISSUED financial and identity fields are immutable.
255. A bill is not income and creates no ledger entry. Only confirmed collections create contribution and ledger movements.
256. Waiver/concession requires bill.waive, mandatory reason, configurable approval, and cannot reduce payable below confirmed payments.
257. Partial payments are allowed unless disabled by bill policy. Outstanding is derived from confirmed allocations only.
258. Overpayment is rejected by default; controlled excess creates a separate direct contribution only with explicit contributor confirmation.
259. Cancellation of an unpaid issued bill requires reason and audit. A paid or partially paid bill cannot be cancelled; it may be closed or replaced through compensating workflows.
260. Replacement creates a new number and reciprocal links; the original remains visible as REPLACED.
261. PDF generation is asynchronous. A render failure does not cancel the bill and may be retried manually or by bounded worker retry policy.
262. QR code resolves to a short-lived public bill/payment page with safe fields only; payment links never expose internal IDs.
263. WhatsApp/email/print delivery records template version, locale, provider message ID, actor, and delivery status.
264. Batch generation is idempotent, resumable, bounded, and produces a manifest of generated/skipped/failed records.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
enum ContributionBillStatus { DRAFT GENERATED ISSUED PARTIALLY_PAID PAID OVERDUE WAIVED CANCELLED RENDER_FAILED REPLACED }
enum BillAmountMode { FIXED OPEN_AMOUNT }
```

```
model ContributionBill {
  id String @id @default(uuid()) @db.Uuid
  organizationId String @db.Uuid
  eventId String @db.Uuid
  contributorAccountId String @db.Uuid
  assignedVolunteerId String? @db.Uuid
  billNumber String? @unique @db.VarChar(90)
  sequence BigInt?
  calendarYear Int?
  status ContributionBillStatus @default(DRAFT)
  amountMode BillAmountMode @default(FIXED)
  requestedAmountPaise BigInt?
  waiverAmountPaise BigInt @default(0)
  payableAmountPaise BigInt?
  confirmedCollectionPaise BigInt @default(0)
  outstandingAmountPaise BigInt?
  issueDate DateTime?
  dueDate DateTime?
  personalizedMessage String? @db.Text
  languageCode String @default("mr") @db.VarChar(10)
  snapshot Json?
  paymentTokenHash String? @unique
  pdfDocumentId String? @unique @db.Uuid
  templateVersion String? @db.VarChar(60)
  issuedByUserId String? @db.Uuid
  cancelledByUserId String? @db.Uuid
  cancellationReason String? @db.Text
  replacesBillId String? @unique @db.Uuid
  replacedByBillId String? @unique @db.Uuid
  idempotencyKey String @unique @db.VarChar(120)
  version Int @default(1)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  lines ContributionBillLine[]
  @@index([organizationId, eventId, status, issueDate])
  @@index([organizationId, eventId, assignedVolunteerId, status])
  @@index([contributorAccountId, createdAt])
}

model ContributionBillLine {
  id String @id @default(uuid()) @db.Uuid
  billId String @db.Uuid
  lineNo Int
  contributionType ContributionType
  description String @db.VarChar(250)
  amountPaise BigInt?
  metadata Json?
  bill ContributionBill @relation(fields: [billId], references: [id], onDelete: Cascade)
  @@unique([billId, lineNo])
}

model BillGenerationJob {
  id String @id @default(uuid()) @db.Uuid
  organizationId String @db.Uuid
  eventId String @db.Uuid
  requestedByUserId String @db.Uuid
  filters Json
  templateVersion String @db.VarChar(60)
  idempotencyKey String @unique @db.VarChar(120)
  status JobStatus @default(QUEUED)
  totalCount Int @default(0)
  generatedCount Int @default(0)
  skippedCount Int @default(0)
  failedCount Int @default(0)
  manifestDocumentId String? @db.Uuid
  createdAt DateTime @default(now())
}
```

3. API Specifications

Method	Route	Purpose	Role Gate
GET	/api/v1/events/:eventId/contribution-bills	List/filter bills.	bill.view
POST	/api/v1/events/:eventId/contribution-bills	Create draft bill.	bill.create
PATCH	/api/v1/contribution-bills/:id	Update draft.	bill.update
POST	/api/v1/contribution-bills/:id/generate	Freeze preview and queue PDF.	bill.generate
POST	/api/v1/contribution-bills/:id/issue	Allocate number, freeze snapshot, issue.	bill.issue
POST	/api/v1/events/:eventId/contribution-bill-jobs	Queue filtered batch generation.	bill.batch-generate
GET	/api/v1/contribution-bill-jobs/:id	Read batch progress/manifest.	bill.batch-generate
POST	/api/v1/contribution-bills/:id/share	Send WhatsApp/email delivery.	bill.share
POST	/api/v1/contribution-bills/:id/waive	Approve waiver/concession.	bill.waive
POST	/api/v1/contribution-bills/:id/cancel	Cancel unpaid issued bill.	bill.cancel
POST	/api/v1/contribution-bills/:id/replace	Issue linked replacement.	bill.replace
POST	/api/v1/contribution-bills/:id/render-retry	Retry failed PDF.	bill.manage
GET	/api/v1/public/contribution-bills/:token	Safe bill/payment page.	Public

4. Domain Events

Subject	Payload Minimum
pauti.contribution.bill.issued.v1	organizationId, eventId, billId, contributorAccountId, assignedVolunteerId, billNumber, payableAmountPaise, snapshotVersion
pauti.contribution.bill.status-changed.v1	billId, fromStatus, toStatus, confirmedCollectionPaise, outstandingAmountPaise
pauti.contribution.bill.render-requested.v1	billId, templateVersion, languageCode, snapshotChecksum
pauti.contribution.bill.cancelled.v1	billId, reasonCode, actorUserId, cancelledAt

5. Error Codes & Validation

Code	Meaning
BILL_NOT_EDITABLE	Only DRAFT bills may change business fields.
BILL_SEQUENCE_CONFLICT	Atomic sequence or number uniqueness failed; transaction must retry safely.
BILL_OVERPAYMENT	Requested allocation exceeds current outstanding amount.
BILL_WAIVER_BELOW_PAID	Waiver would make payable less than confirmed collections.
BILL_CANCEL_HAS_COLLECTIONS	Partially/fully paid bill cannot be cancelled.
BILL_TOKEN_INVALID_OR_EXPIRED	Public token is invalid, expired, revoked, or tenant scope is wrong.

6. Verification & Testing Tasks

- Fifty concurrent issues produce unique, gap-tolerant, monotonically increasing event-year bill numbers.
- Batch replay with the same idempotency key creates no duplicate bills, jobs, PDFs, or deliveries.
- Contributor edits after issue do not change bill PDF, public page, or audit snapshot.
- Open-amount, fixed, waiver, partial payment, overdue, cancellation, replacement, and render-failure paths are covered.
- Public page exposes no internal UUID, phone, email, private address, provider secret, or unrestricted document URL.

Module Specification: Payment Collection Records

Overview & Metadata

Module Path	apps/contribution-service/src/modules/collection-records/
Default Assignee/Role	Unassigned - Finance Domain
Initial Status	Pending
Core Responsibility	Bill-linked and direct payment collection records, verification, cheque clearance, evidence, allocation, collector settlement, reconciliation, cancellation, and exact financial side effects.

1. Functional Requirements & Business Logic

- 270. Create a collection against one issued bill or as a direct contribution. A collection always references an identified contributor and target event.
- 271. Support CASH, UPI, BANK_TRANSFER, CHEQUE, RAZORPAY, CARD, and OTHER modes. Provider-specific references are validated and normalized.
- 272. Capture collector/volunteer, collectedAt, amount, currency, location, optional geolocation, payment reference, evidence document, notes, and source channel.
- 273. Lifecycle is RECORDED -> PENDING_VERIFICATION -> CONFIRMED, with PENDING_CHEQUE_CLEARANCE, REJECTED, CANCELLED, CHEQUE_BOUNCED, PARTIALLY_REFUNDED, and REFUNDED.
- 274. Manual cash, UPI, bank, cheque, and card collections require collection.confirm unless an organization policy permits trusted collector auto-confirmation below a threshold.
- 275. Cheque creates no contribution confirmation, receipt, or positive ledger entry until cleared. Bounce preserves evidence and updates status.
- 276. Bill allocation cannot exceed current outstanding amount. Open-amount bills accept any positive configured amount.
- 277. The confirmation transaction locks bill allocation state, confirms the contribution payment, updates bill totals, writes outbox events, and records an idempotent finance posting request.
- 278. Receipt allocation occurs after financial confirmation and is exactly-once by collection ID.
- 279. Confirmed records are immutable. Correction requires cancellation/refund/compensating records; destructive delete is prohibited.
- 280. Collector and contributor snapshots are frozen at confirmation. Later reassignment or profile edits do not alter the collection history.
- 281. Cash collectors submit daily settlement/handover. Expected cash is derived from confirmed cash collections less approved refunds and prior handovers.
- 282. Settlement variance requires reason, approver, and audit. Settlement does not duplicate ledger income; it transfers custody between asset accounts when configured.
- 283. Offline/mobile submissions use client-generated idempotency keys and server timestamps; clock skew and duplicate sync are handled safely.
- 284. Razorpay frontend success never confirms a collection; only signed server verification or webhook may confirm.

2. Data Model Schema (PostgreSQL / Prisma ORM)

```
enum CollectionStatus { RECORDED PENDING_VERIFICATION PENDING_CHEQUE_CLEARANCE CONFIRMED REJECTED CANCELLED CHEQUE_BOUNCED PARTIALLY_REFUNDED RECORDED }
enum CollectionMode { CASH UPI BANK_TRANSFER CHEQUE RAZORPAY CARD OTHER }
enum CollectionSource { ADMIN_WEB COLLECTOR_WEB COLLECTOR_MOBILE PUBLIC_PAYMENT IMPORT_API }
```

```
model CollectionRecord {
  id String @id @default(uuid()) @db.Uuid
  organizationId String @db.Uuid
  eventId String @db.Uuid
  contributorAccountId String @db.Uuid
  billId String? @db.Uuid
  contributionId String? @db.Uuid
  collectorVolunteerId String? @db.Uuid
  status CollectionStatus @default(RECORDED)
  mode CollectionMode
  source CollectionSource
  amountPaise BigInt
  currency String @default("INR") @db.Char(3)
  collectedAt DateTime
  paymentReference String? @db.VarChar(160)
  providerPaymentId String? @unique @db.VarChar(160)
  chequeSnapshot Json?
  locationSnapshot Json?
  evidenceDocumentId String? @db.Uuid
  contributorSnapshot Json?
  collectorSnapshot Json?
  verifiedByUserId String? @db.Uuid
  verifiedAt DateTime?
  rejectionReason String? @db.Text
  receiptId String? @unique @db.Uuid
  idempotencyKey String @unique @db.VarChar(120)
  createdByUserId String @db.Uuid
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  allocations CollectionAllocation[]
  @@index([organizationId, eventId, status, collectedAt])
  @@index([organizationId, eventId, collectorVolunteerId, collectedAt])
  @@index([billId, status])
}
```

```
model CollectionAllocation {
  id String @id @default(uuid()) @db.Uuid
  collectionRecordId String @db.Uuid
  billId String? @db.Uuid
  contributionId String @db.Uuid
  amountPaise BigInt
  createdAt DateTime @default(now())
  collection CollectionRecord @relation(fields: [collectionRecordId], references: [id], onDelete: Restrict)
  @@index([billId, createdAt])
  @@unique([collectionRecordId, contributionId, billId])
}
```

```
model CollectorSettlement {
  id String @id @default(uuid()) @db.Uuid
  organizationId String @db.Uuid
  eventId String @db.Uuid
  collectorVolunteerId String @db.Uuid
  settlementDate DateTime
  expectedCashPaise BigInt
  handedOverCashPaise BigInt
  variancePaise BigInt
  status SettlementStatus @default(SUBMITTED)
  reason String? @db.Text
  submittedByUserId String @db.Uuid
  approvedByUserId String? @db.Uuid
  idempotencyKey String @unique @db.VarChar(120)
  createdAt DateTime @default(now())
  @@unique([organizationId, eventId, collectorVolunteerId, settlementDate])
}
```

3. API Specifications

Method	Route	Purpose	Role Gate
GET	/api/v1/events/:eventId/collection-records	List/filter collection records.	collection.view
POST	/api/v1/events/:eventId/collection-records	Record bill/direct collection.	collection.create
GET	/api/v1/collection-records/:id	Read record, allocation, verification, receipt.	collection.view
POST	/api/v1/collection-records/:id/verify	Confirm manual collection.	collection.confirm
POST	/api/v1/collection-records/:id/reject	Reject with reason.	collection.confirm
POST	/api/v1/collection-records/:id/cheque-clear	Confirm cheque clearance.	collection.confirm
POST	/api/v1/collection-records/:id/cheque-bounce	Record cheque bounce.	collection.confirm
POST	/api/v1/collection-records/:id/cancel	Cancel unconfirmed record.	collection.cancel
POST	/api/v1/collector-settlements	Submit daily cash handover.	collection.settle
POST	/api/v1/collector-settlements/:id/approve	Approve/reject settlement.	collection.settlement.approve
GET	/api/v1/collectors/:volunteerId/daily-summary	Expected/confirmed/settled/variance summary.	collection.view

4. Confirmation Saga

```

collection recorded
-> manual/provider verification
-> contribution payment confirmed (local transaction)
-> bill confirmedCollection/outstanding updated (same DB transaction)
-> outbox: collection.confirmed.v1
-> finance-service posts balanced ledger transaction (idempotent)
-> receipt allocated exactly once by collectionId
-> PDF render job queued
-> WhatsApp/email/in-app notification queued
-> reporting/public read models updated

```

5. Error Codes

Code	Meaning
COLLECTION_AMOUNT_INVALID	Amount must be positive integer paise and within configured limits.
COLLECTION_BILL_NOT_PAYABLE	Bill is cancelled, replaced, waived/closed, or has no payable balance.
COLLECTION_ALLOCATION_EXCEEDS_OUTSTANDING	Allocation exceeds outstanding bill amount.
COLLECTION_ALREADY_CONFIRMED	Immutable confirmed collection cannot be confirmed or edited again.
COLLECTION_CHEQUE_NOT_CLEARED	Cheque cannot create receipt or ledger credit before clearance.
COLLECTION_COLLECTOR_NOT_AUTHORIZED	Volunteer has no active assignment/permission for the event or scope.

6. Verification & Testing Tasks

- Concurrent partial payments cannot over-allocate the same bill; row/version locking and constraints are tested.
- Duplicate mobile sync, webhook replay, and API retry produce one collection, one ledger posting, and one receipt.
- Cheque pending/bounce paths create no positive ledger or receipt.
- Collector suspension between record and verification is handled by policy without losing evidence.
- Settlement expected cash reconciles exactly to confirmed cash collections and approved refunds.
- Audit and logs never contain full mobile, email, bank account, cheque image URL, or provider secrets.

Module Specification: Unified Contribution Records

Overview & Metadata

Module Path	apps/contribution-service/src/modules/contributions/
Default Assignee/Role	Unassigned - Finance Domain
Initial Status	Pending
Core Responsibility	Unified financial contribution aggregate for personalized bill collections, direct donations, sponsorships, advertisements, memberships, grants, and installments, with compatibility migration from the existing donations module.

1. Functional Requirements & Business Logic

- 295. Replace the business term Donation aggregate with ContributionRecord while retaining donation API aliases during a documented deprecation window.
- 296. Contribution source types are BILL_COLLECTION, DIRECT, SPONSORSHIP, ADVERTISEMENT, MEMBERSHIP, GOVERNMENT_GRANT, INSTITUTIONAL_GRANT, and IMPORTED_LEGACY.
- 297. Every confirmed collection allocation belongs to exactly one contribution record and increments confirmedAmountPaise exactly once.
- 298. Bill-linked contribution requested amount is derived from the bill snapshot; direct contributions may be open or pledged.
- 299. Status is DRAFT, PENDING, PARTIALLY_RECEIVED, CONFIRMED, RECEIPT_ISSUED, CANCELLED, PARTIALLY_REFUNDED, or REFUNDED.
- 300. Confirmed amount equals the sum of confirmed non-cancelled allocations less successful refunds; no editable aggregate total is accepted from clients.
- 301. Contribution confirmation, ledger request, receipt request, and outbox messages use idempotency keys and transactional boundaries.
- 302. Manual confirmed collections are non-refundable by default; eligible online duplicates/incorrect payments follow payment-service refund policy.
- 303. Cancellation of an unconfirmed contribution requires reason. Confirmed financial movements use compensating entries, not mutation.
- 304. Contributor, event, organization, bill, collector, purpose, privacy, and payment snapshots are retained for historical reporting.
- 305. Public visibility respects contributor privacy; contact data is never included in public events or standard exports.
- 306. Legacy donation IDs remain stable and map one-to-one to migrated contribution IDs.

2. Data Model Schema

```
enum ContributionSourceType { BILL_COLLECTION DIRECT SPONSORSHIP ADVERTISEMENT MEMBERSHIP GOVERNMENT_GRANT INSTITUTIONAL_GRANT IMPORTED_LEGACY }
enum ContributionStatus { DRAFT PENDING PARTIALLY_RECEIVED CONFIRMED RECEIPT_ISSUED CANCELLED PARTIALLY_REFUNDED REFUNDED }

model ContributionRecord {
  id String @id @default(uuid()) @db.Uuid
  legacyDonationId String? @unique @db.Uuid
  organizationId String @db.Uuid
  eventId String @db.Uuid
  contributorAccountId String @db.Uuid
  sourceType ContributionSourceType
  status ContributionStatus @default(DRAFT)
  billId String? @db.Uuid
  pledgedAmountPaise BigInt?
  confirmedAmountPaise BigInt @default(0)
  refundedAmountPaise BigInt @default(0)
  purpose String? @db.VarChar(250)
  publicVisibility PublicVisibility @default(NAME_AND_AMOUNT)
  snapshot Json?
  idempotencyKey String @unique @db.VarChar(120)
  createdByUserId String @db.Uuid
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  @@index([organizationId, eventId, status, createdAt])
  @@index([contributorAccountId, createdAt])
  @@index([billId])
}
```

3. Compatibility & Migration

Area	Enterprise Requirement
Database	Online expand-and-contract migration; backfill contribution rows with stable IDs and checksums.
API	Existing /donations routes remain read/write aliases for one deprecation period and emit Deprecation/Sunset headers.
Events	Publish new contribution subjects; dual-publish legacy donation subjects only during controlled migration.
Reports	Reconcile migrated totals to legacy donation, installment, receipt, and ledger records before cutover.
Rollback	Migration is reversible before final write cutover; no destructive rename in the same release.

4. Verification & Testing Tasks

- Legacy and new APIs produce identical financial results during compatibility period.
- Backfill counts, sums, receipts, ledger links, and public visibility reconcile exactly.
- Event redelivery cannot double-increment confirmedAmountPaise.

Module Specification Extension: Contribution Receipts

Overview & Metadata

Module Path	apps/contribution-service/src/modules/receipts/
Default Assignee/Role	Unassigned - Documents & Finance
Initial Status	Pending
Core Responsibility	Immutable multilingual receipts for each confirmed contribution payment, including bill, collection, contributor, collector, payment, balance, PDF, QR verification, sharing, cancellation, refund state, and replacement.

1. Functional Requirements & Business Logic

- 311. Generate one immutable receipt for every confirmed collection allocation and every verified in-kind contribution.
- 312. Partial payments generate separate receipt numbers. Each receipt shows current payment, cumulative confirmed amount, and remaining bill balance snapshot.
- 313. Freeze organization, event, contributor, bill number, collection reference, amount, amount in words, payment mode/reference, collector, signature, locale, and numbering.
- 314. Receipt number allocation remains atomic at event-year scope and occurs exactly once per collection allocation.
- 315. Receipt financial record commits before asynchronous PDF rendering. Render failure sets RENDER_FAILED and is retryable without issuing a new number.
- 316. PDF and public verification page include a QR token with a stored hash, revocation state, and safe verification data only.
- 317. Receipts cannot be edited or deleted. Corrections require formal cancellation and optional replacement with reciprocal references.
- 318. Cancellation requires separate approval, reason, original snapshot, and compensating financial workflow where applicable.
- 319. Refunded online collections update receipt verification status to PARTIALLY_REFUNDED or REFUNDED; original PDF remains available with a visible status marker.
- 320. WhatsApp/email delivery sends the immutable PDF or short-lived signed URL and public verification URL; provider delivery status is audited.
- 321. Templates are versioned and support English, Marathi, Hindi, or trilingual output. Glyph, line-break, and amount-in-words tests are mandatory.
- 322. No mutable contributor, volunteer, event, organization, template, or settings value is fetched during re-render; the frozen snapshot is authoritative.

2. Extended Data Model

```
enum ReceiptSourceType { CONTRIBUTION_COLLECTION IN_KIND_CONTRIBUTION LEGACY_DONATION_INSTALLMENT }
enum ReceiptStatus { GENERATING ISSUED CANCELLED PARTIALLY_REFUNDED REFUNDED RENDER_FAILED REPLACED }

model Receipt {
  id String @id @default(uuid()) @db.Uuid
  organizationId String @db.Uuid
  eventId String @db.Uuid
  receiptBookId String @db.Uuid
  sourceType ReceiptSourceType
  contributionId String? @db.Uuid
  collectionRecordId String? @unique @db.Uuid
  billId String? @db.Uuid
  contributorAccountId String @db.Uuid
  collectorVolunteerId String? @db.Uuid
  receiptNumber String @unique @db.VarChar(90)
  sequence BigInt
  calendarYear Int
  status ReceiptStatus @default(GENERATING)
  amountPaise BigInt?
  cumulativeConfirmedPaise BigInt?
  remainingBillBalancePaise BigInt?
  verificationTokenHash String @unique
  snapshot Json
  snapshotChecksum String @db.VarChar(128)
  pdfDocumentId String? @unique @db.Uuid
  templateVersion String @db.VarChar(60)
  issuedByUserId String @db.Uuid
  issuedAt DateTime?
  replacesReceiptId String? @unique @db.Uuid
  replacedByReceiptId String? @unique @db.Uuid
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  @@index([organizationId, eventId, issuedAt])
  @@index([billId, issuedAt])
  @@index([collectorVolunteerId, issuedAt])
}
```

3. API Specifications

Method	Route	Purpose	Role Gate
GET	/api/v1/receipts/:id	Retrieve immutable receipt metadata.	receipt.view or owning contributor
GET	/api/v1/receipts/:id/pdf	Get short-lived signed PDF URL.	receipt.view or owning contributor
POST	/api/v1/receipts/:id/share	Share by WhatsApp/email.	receipt.share or owning contributor
POST	/api/v1/receipts/:id/cancel	Cancel with separate approval.	receipt.cancel
POST	/api/v1/receipts/:id/replace	Issue linked replacement.	receipt.replace
POST	/api/v1/receipts/:id/render-retry	Retry same snapshot/template.	receipt.manage
GET	/api/v1/public/receipts/verify/:token	Safe QR verification.	Public

4. Verification & Testing Tasks

- Every confirmed partial payment creates exactly one receipt with unique event-year sequence.
- Receipt amount, cumulative paid, remaining balance, bill number, collector, and payment reference match confirmation snapshots.
- Template/settings/master-data changes cannot alter historical PDF or verification results.
- Cancellation, replacement, partial refund, full refund, render retry, and provider delivery failure are covered.
- QR verification is rate-limited and reveals no private contact, internal IDs, or permanent document URL.

End-to-End Personalized Collection Workflow

Step	Owning Service	Transaction / Evidence
1. Register/import contributor account	donor-service	Validated profile, address/category, duplicate result, audit.
2. Register volunteer and event assignment	volunteer-service	Active assignment, scope, effective period, audit.
3. Create/generate personalized bill	contribution-service	Draft lines, contributor/collector references, idempotency.
4. Issue bill	contribution-service	Atomic bill number, immutable snapshot, payment token, outbox.
5. Render/share bill	document + notification	Versioned PDF, signed URL, delivery evidence.
6. Record/receive payment	contribution + payment	Collection evidence, provider/manual verification state.
7. Confirm collection	contribution-service	Locked allocation, contribution total, bill balance, outbox.
8. Post ledger	finance-service	Balanced idempotent double-entry transaction.
9. Allocate/render receipt	contribution + document	Atomic receipt number, immutable snapshot/PDF/QR.
10. Notify and report	notification + reporting	Delivery result, collector/bill/contribution/receipt read models.
11. Settle collector cash	contribution + finance	Expected/handed-over/variance, custody transfer, approval.

Required RBAC Permission Additions

Domain	Permissions
Volunteer	volunteer.view, create, update, manage, sensitive.view, import, assignment.manage
Contributor	contributor.view, create, update, assign, merge, import
Bill	bill.view, create, update, generate, issue, batch-generate, share, waive, cancel, replace, manage
Collection	collection.view, create, confirm, cancel, settle, settlement.approve, export
Contribution	contribution.view, create, confirm, cancel, refund.view, export
Receipt	receipt.view, share, cancel, replace, manage, export

System-Wide Acceptance Criteria Additions

- Volunteer and contributor records are tenant-isolated at gateway, service, repository, cache, event, export, file, and database-policy layers.
- A volunteer without an active event/scope assignment cannot collect or access assigned contributor portfolios.
- Issued bill numbers are unique under at least 50 concurrent issue requests and are never reused after cancellation or rollback.

- Issued bill snapshots remain unchanged after contributor, volunteer, event, organization, settings, or template changes.
- Concurrent partial collections cannot overpay or create negative outstanding balance.
- Every confirmed collection creates exactly one contribution increment, one balanced ledger transaction, and one receipt allocation.
- Cheque pending/bounced collections create no confirmed income, ledger credit, or receipt.
- Duplicate API requests, mobile sync, NATS redelivery, worker retry, and payment webhook replay create no duplicate financial side effects.
- Collector daily summaries and settlements reconcile to confirmed cash collections, approved refunds, and custody-transfer ledger entries.
- Public bill and receipt pages expose only approved safe fields and use hashed tokens, expiry/revocation, throttling, and signed document access.
- English, Marathi, Hindi, and trilingual bills/receipts pass glyph, amount-in-words, pagination, and QR verification tests.
- Reports include volunteer performance, bill register/outstanding, collections by mode/collector/area, contribution register, and receipt register; all reconcile to source and ledger data.
- Migration from donations to contributions preserves IDs, totals, receipts, ledger references, public visibility, and audit evidence with documented rollback.
- All new services/modules include migrations, OpenAPI/AsyncAPI/Protobuf contracts, runbooks, dashboards, alerts, SLOs, tests, and rollback evidence before acceptance.

Mandatory Critical Journey Tests

- Contributor import -> volunteer assignment -> batch bill issue -> PDF/WhatsApp delivery -> partial cash collection -> verification -> ledger -> receipt -> settlement.
- Public QR/Razorpay payment -> signed webhook -> contribution confirmation -> bill paid -> receipt PDF/QR -> notification; replay all events and webhooks.
- Cheque record -> pending clearance -> bounce; verify zero confirmed amount, zero positive ledger, zero receipt.
- Concurrent collectors submit against one bill; only valid outstanding allocations commit and duplicates are idempotent.
- Contributor/volunteer edited after issue; historical bill and receipt snapshots remain byte-stable except approved status watermark derivatives.
- Cross-tenant bill/collection/receipt/document ID substitution, token tampering, cache collision, and export request return 403/404 without leakage.

Revision 2.1 Completion Definition

Definition of Done

The revision is complete only when the updated services are independently buildable and deployable, database migrations and compatibility adapters are verified, all critical journeys pass Playwright and Testcontainers pipelines, finance reconciliation is exact, security/privacy tests pass, generated documents render correctly in all supported languages, and operational dashboards/runbooks are approved.